COMPREHENSIVE SOURCE CODE DOCUMENT

Faculty Feedback & Appraisal System

Complete Production Source Codebase — Backend, Database & Frontend

GitHub Repository: `https://github.com/ayushgautam50605-ship-it/faculty-feedback-system`

Default Branch: `main`

Architecture: `Node.js, Express REST API, SQLite (better-sqlite3), React 18, Vite, Tailwind CSS`

Department / Cohort: `Information Technology (Semester 5)`

Total Modules: `49 Source Files`

SYSTEM PURPOSE & METHODOLOGY

This document provides the full, unabridged source code for the Faculty Feedback & Appraisal System. The platform digitizes institutional course-end surveys, replaces manual evaluation forms, ensures anonymous student submissions, and calculates transparent weighted faculty appraisal scores across 10 evaluation criteria. It equips Heads of Department (HOD) with real-time academic analytics, completion gauges, faculty rankings, and report exports.

Express.js API | better-sqlite3 | JWT & Bcrypt RBAC | React 18 SPA | Tailwind CSS | Recharts Data Visualizations | Self-Registration

Table of Contents (49 Source Files)

CHAPTER 1: ARCHITECTURE & SYSTEM SPECIFICATION

1. System Architecture & Design Document — System Design
   `ARCHITECTURE.md`
2. Project Readme & Operational Guide — Documentation
   `README.md`

CHAPTER 2: DATABASE LAYER & INITIALIZATION

3. Database Configuration & 8-Table Normalized Schema — Database Layer
   `backend/config/db.js`

CHAPTER 3: CORE APPRAISAL ENGINE & SECURITY MIDDLEWARE

4. Appraisal Scoring & Mathematical Grade Engine — Business Logic
   `backend/utils/appraisal.js`
5. JWT Authentication & Role Authorization Middleware — Security / RBAC
   `backend/middleware/auth.js`
6. Express Server Entrypoint & API Dispatcher — Server Core
   `backend/server.js`

CHAPTER 4: BACKEND CONTROLLERS (BUSINESS LOGIC)

7. Authentication & Student Self-Registration Controller — Controller
   `backend/controllers/authController.js`
8. Feedback Submission & Validation Controller — Controller
   `backend/controllers/feedbackController.js`
9. Analytics & KPI Aggregation Controller — Controller
   `backend/controllers/analyticsController.js`
10. Faculty Dashboard Performance Controller — Controller
    `backend/controllers/facultyController.js`
11. Appraisal Management Controller — Controller
    `backend/controllers/appraisalController.js`
12. System Administration Controller — Controller
    `backend/controllers/adminController.js`

CHAPTER 5: BACKEND ROUTE DEFINITIONS

13. Authentication Routes — API Routes
    `backend/routes/authRoutes.js`
14. Feedback Routes — API Routes
    `backend/routes/feedbackRoutes.js`
15. Faculty Routes — API Routes
    `backend/routes/facultyRoutes.js`

16. Analytics Routes — API Routes
`backend/routes/analyticsRoutes.js`

17. Appraisal Routes — API Routes
`backend/routes/appraisalRoutes.js`

18. Admin Management Routes — API Routes
`backend/routes/adminRoutes.js`

CHAPTER 6: DATA SEEDING & DEPLOYMENT SCRIPTS

19. IT Faculty & Subjects Provisioning — Scripts
`backend/add_faculty.js`

20. IT Student Cohort Registration — Scripts
`backend/add_students.js`

21. Database Reset & Clean Utility — Scripts
`backend/clean.js`

22. Mock Data Seeder — Scripts
`backend/seed.js`

CHAPTER 7: FRONTEND APPLICATION CORE & STATE

23. Vite HTML Entrypoint — Frontend Core
`frontend/index.html`

24. React Root Mount — Frontend Core
`frontend/src/main.jsx`

25. Application Routes & Role Protection — Frontend Core
`frontend/src/App.jsx`

26. Axios HTTP Client & Auth Interceptor — Frontend Core
`frontend/src/api/client.js`

27. Global Authentication Context — Frontend Core
`frontend/src/context/AuthContext.jsx`

28. Tailwind CSS Global Styles — Frontend Core
`frontend/src/index.css`

CHAPTER 8: FRONTEND REUSABLE COMPONENTS

29. App Layout & Navigation Sidebar — UI Component
`frontend/src/components/Layout.jsx`

30. KPI Card Display — UI Component
`frontend/src/components/KPICard.jsx`

31. Star & Numeric Rating Input — UI Component
`frontend/src/components/RatingInput.jsx`

32. Protected Route Barrier — UI Component
`frontend/src/components/ProtectedRoute.jsx`

33. Empty State Widget — UI Component
`frontend/src/components/EmptyState.jsx`

34. Toast Notification Widget — UI Component

frontend/src/components/Toast.jsx

CHAPTER 9: STUDENT PORTAL PAGES

35. Student Dashboard — Student Page
frontend/src/pages/student/StudentDashboard.jsx

36. Interactive Feedback Submission Form — Student Page
frontend/src/pages/student/FeedbackForm.jsx

CHAPTER 10: FACULTY PORTAL PAGES

37. Faculty Analytics & Appraisal Dashboard — Faculty Page
frontend/src/pages/faculty/FacultyDashboard.jsx

CHAPTER 11: HOD & ADMINISTRATION PAGES

38. HOD Executive Analytics Dashboard — HOD Page
frontend/src/pages/hod/HodDashboard.jsx

39. Faculty Roster & Assignment Page — HOD Page
frontend/src/pages/hod/HodFacultyPage.jsx

40. Student Roster Management Page — HOD Page
frontend/src/pages/hod/HodStudentsPage.jsx

41. Faculty Appraisals & Scoring Engine Page — HOD Page
frontend/src/pages/hod/HodAppraisalsPage.jsx

42. Feedback Period & Criteria Settings — HOD Page
frontend/src/pages/hod/HodSettingsPage.jsx

43. Reports & CSV Export Page — HOD Page
frontend/src/pages/hod/HodReportsPage.jsx

44. Unified Login & Student Self-Registration Page — Auth Page
frontend/src/pages/Login.jsx

CHAPTER 12: BUILD & DEPENDENCY CONFIGURATION

45. Root Workspace Package Config — Config
package.json

46. Backend Package & Dependencies — Config
backend/package.json

47. Frontend Package & Dependencies — Config
frontend/package.json

48. Vite Bundler Configuration — Config
frontend/vite.config.js

49. Tailwind CSS Design System Config — Config
frontend/tailwind.config.js

CHAPTER 1: ARCHITECTURE & SYSTEM SPECIFICATION

SYSTEM DESIGN

1. System Architecture & Design Document

📁 Source File: `ARCHITECTURE.md`

Architecture diagrams, entity relationships, database schema specifications, appraisal mathematical formulation, and API contracts.

```
# System Architecture & Design Document

## 1. Problem Statement & Proposed Solution

Traditional semester-end faculty feedback is often paper-based or a bare
Google Form: aggregation is manual, there is no standardized scoring
method, and the HOD has no real visibility into trends across faculty,
subjects or semesters. This system centralizes collection, automatically
aggregates and analyzes responses, and turns raw ratings into an
auditable **appraisal score** with real analytics — without ever
exposing which student said what.

## 2. System Architecture

```
┌──────────────────┐        HTTPS/JSON        ┌──────────────────┐        SQL        ┌────────────┐
│   React SPA      │ ─────────────────────▶ │  Express REST API  │ ─────────────▶ │  SQLite DB │
│  (Vite, Tailwind,  │ ◀─────────────────────  │  (JWT auth, role-  │ ◀─────────────  │  (better-  │
│   Recharts)      │          JSON            │   based middleware) │                 │  sqlite3)  │
└──────────────────┘                          └──────────────────┘                 └────────────┘
       │                                              │
       │  localStorage: JWT + user profile            │  bcrypt password hashing
       ▼                                              ▼
  Role-based routing                          appraisal.js — scoring engine
  (student / faculty / hod)                    (pure functions, no side effects
                                                beyond the appraisal table)
```

- **Frontend** is a single-page app. Route access is gated client-side by
  `ProtectedRoute` (role check) and server-side by `authenticate` +
  `authorize(...roles)` middleware on every sensitive endpoint — the UI
  gate is a convenience, the API gate is the real security boundary.
- **Backend** follows a classic layered structure: `routes` (HTTP
  wiring) → `controllers` (request handling, validation) → `config/db.js`
  (data access) with a separate `utils/appraisal.js` holding the scoring
  logic so it can be unit-tested independently and reused by both the
  faculty-performance endpoint and the HOD appraisal endpoints.
- **Database** is SQLite for zero-config local/demo use; the schema is
  fully normalized ANSI SQL and would move to Postgres/MySQL with no
  structural changes — only the driver in `config/db.js` changes.

3. Database Schema (Entity Relationship)

```mermaid
erDiagram
    USERS ||--o| STUDENTS : "has profile"
    USERS ||--o| FACULTY : "has profile"
    FACULTY ||--o{ SUBJECTS : teaches
    STUDENTS ||--o{ FEEDBACK : submits
    FACULTY ||--o{ FEEDBACK : "is rated in"
    SUBJECTS ||--o{ FEEDBACK : "is rated for"
    FEEDBACK_PERIODS ||--o{ FEEDBACK : contains
    FEEDBACK ||--|{ FEEDBACK_RATINGS : has
    CRITERIA ||--o{ FEEDBACK_RATINGS : "scored against"
    FACULTY ||--o{ APPRAISAL : "generates"

    USERS {
        int id PK
        string name
        string email UK
        string password_hash
        string role "student|faculty|hod"
        string department
    }
    STUDENTS {
        int id PK
        int user_id FK
        string department
        int semester
```
```

```
        string section
        string roll_number
    }
    FACULTY {
        int id PK
        int user_id FK
        string department
        string designation
    }
    SUBJECTS {
        int id PK
        string subject_name
        string subject_code
        int faculty_id FK
        string department
        int semester
    }
    CRITERIA {
        int id PK
        string name UK
        real weight
        bool is_active
    }
    FEEDBACK_PERIODS {
        int id PK
        string academic_year
        int semester
        bool is_open
        bool allow_editing
    }
    FEEDBACK {
        int id PK
        int student_id FK
        int faculty_id FK
        int subject_id FK
        int period_id FK
        string comments
        datetime submitted_at
    }
    FEEDBACK_RATINGS {
        int id PK
        int feedback_id FK
        int criterion_id FK
        int rating "1-5"
    }
    APPRAISAL {
        int id PK
        int faculty_id FK
        int semester
        string academic_year
        int total_responses
        real average_rating
        real appraisal_score
        string grade
    }
```

Key integrity rules enforced at the database level:
- `UNIQUE(student_id, faculty_id, subject_id, period_id)` on `feedback` — a student cannot submit duplicate feedback for the same faculty/subject in the same collection period.
- `CHECK (rating BETWEEN 1 AND 5)` on `feedback_ratings`.
- `ON DELETE CASCADE` from `users` down through profiles, subjects and feedback, so removing a faculty/student account cleans up correctly.

4. User Flows

Student

```
Login → Student Dashboard (subject list + completion bar)
      → Select subject → Rate every criterion (1–5) → optional comment
      → Submit → Confirmation screen → back to subject list
```

Duplicate submission is blocked unless the HOD has enabled editing for the open period.

Faculty

```
Login → My Performance
      → KPI cards (overall rating, appraisal score, responses, semester)
```

````
      → Radar chart (criterion-wise) · Trend line (semester-over-semester)
      → Rating distribution · How-it's-calculated panel · Anonymous comments
```
Faculty can only ever query their **own** `faculty_id` — enforced in
`facultyController.getPerformance` and `appraisalController.getAppraisal`,
independent of anything the frontend does.

### HOD / Admin
```
Login → Dashboard (KPIs, faculty comparison, criteria radar, semester trend, activity)
      → Faculty (list → drill into any faculty's full performance page)
      → Students (add/remove)
      → Appraisals (regenerate for a semester, ranked table with grade)
      → Reports (CSV export)
      → Settings (toggle criteria, edit the appraisal formula, open/close feedback window)
```

## 5. Page / Screen Structure

| Route | Role | Purpose |
|---|---|---|
| `/login` | public | Sign in |
| `/student` | student | Subject list + inline feedback form |
| `/faculty` | faculty | Own performance dashboard |
| `/hod` | hod | Main analytics dashboard |
| `/hod/faculty` | hod | Faculty list + management + drilldown into `/faculty`-style view |
| `/hod/students` | hod | Student list + management |
| `/hod/appraisals` | hod | Ranked appraisal table, regenerate button |
| `/hod/reports` | hod | CSV export |
| `/hod/settings` | hod | Criteria, formula config, feedback period control |

## 6. Feature List

- Role-based auth (student / faculty / HOD) with JWT + bcrypt
- Feedback submission with 10 configurable criteria, 1–5 scale, optional comments
- Duplicate-submission prevention, admin-toggleable editing window
- Automatic, transparent, configurable appraisal scoring
- HOD analytics: KPIs, faculty comparison bar chart, criteria radar,
  semester trend line, recent activity feed, department filter
- Faculty performance profile: radar, trend, rating distribution, anonymized comments
- Admin: manage faculty/students/subjects, configure criteria weights,
  open/close feedback periods, edit appraisal formula thresholds
- CSV report export
- Fully responsive (desktop sidebar, mobile bottom nav), loading/empty/error states throughout

## 7. API Architecture

All responses follow `{ success: boolean, message?, data? }`. All
non-auth routes require `Authorization: Bearer <token>`; role checks are
enforced server-side per route.

```
POST   /api/auth/login
GET    /api/auth/me

GET    /api/feedback/criteria          (student, faculty, hod)
GET    /api/feedback/targets           (student)
GET    /api/feedback/status            (student)
POST   /api/feedback                   (student)

GET    /api/faculty                    (any authenticated role)
GET    /api/faculty/:id
GET    /api/faculty/:id/performance    (faculty: own only, hod: any)

GET    /api/analytics/dashboard        (hod)
GET    /api/analytics/faculty          (hod)
GET    /api/analytics/criteria         (hod)
GET    /api/analytics/trend            (hod)
GET    /api/analytics/recent-activity  (hod)
GET    /api/analytics/filters          (hod)

GET    /api/appraisal/:facultyId       (faculty: own only, hod: any)
POST   /api/appraisal/generate         (hod)
GET    /api/appraisal/config           (hod)
PUT    /api/appraisal/config           (hod)

GET    /api/admin/students             (hod)
POST   /api/admin/students
DELETE /api/admin/students/:id
POST   /api/admin/faculty
````

```
DELETE /api/admin/faculty/:id
GET    /api/admin/subjects
POST   /api/admin/subjects
DELETE /api/admin/subjects/:id
GET    /api/admin/criteria
POST   /api/admin/criteria
PUT    /api/admin/criteria/:id
GET    /api/admin/periods
POST   /api/admin/periods/open
POST   /api/admin/periods/close
GET    /api/admin/reports/appraisal.csv
```

8. Appraisal Calculation Methodology

The scoring engine (`backend/utils/appraisal.js`) is deliberately simple linear math — no black box — so it can be explained in a viva and audited by the HOD.

Step 1 — Criterion average. For each active criterion, average every rating it received for that faculty member in the given semester/year:

```
criterionAvg_i = Σ(ratings for criterion i) / count(ratings for criterion i)
```

Step 2 — Weighted average rating. Combine all criterion averages using each criterion's configurable weight (defaults to 1.0, i.e. equal weighting):

```
weightedAverage = Σ(criterionAvg_i × weight_i) / Σ(weight_i)
```

Step 3 — Appraisal score. Normalize against the configured rating scale (default 1–5) into a 0–100 score:

```
appraisalScore = (weightedAverage / scaleMax) × 100
```

Step 4 — Grade. Compare the weighted average against two HOD-configurable thresholds (defaults: 3.0 and 4.5):

```
weightedAverage ≥ excellentThreshold  → "Excellent"
weightedAverage ≥ passThreshold       → "Satisfactory"
otherwise                             → "Needs Improvement"
```

The score is recomputed on demand (`GET /api/appraisal/:facultyId`) and persisted via `POST /api/appraisal/generate`, which upserts one row per `(faculty_id, semester, academic_year)` — so historical appraisals are preserved even as new feedback continues to arrive for a *different* semester, giving the semester-trend charts real historical data.

9. Security & Privacy Measures

- Passwords hashed with bcrypt (never stored or returned in plaintext)
- JWT with 8-hour expiry; 401 responses clear client-side auth state
- Every protected route enforces role via server-side middleware, not just UI hiding
- Faculty can only query their own performance/appraisal — enforced by comparing the JWT's `profileId` to the requested `:id`, not by trusting the client
- Feedback responses are never joined back to a student identity in any faculty- or HOD-facing endpoint — only aggregates and anonymized comment text are returned
- Input validation on every write endpoint (required fields, rating range 1–5, valid faculty/subject pairing)
- Parameterized SQL throughout (via `better-sqlite3` prepared statements) — no string-built queries, so no SQL injection surface

CHAPTER 1: ARCHITECTURE & SYSTEM SPECIFICATION

DOCUMENTATION

2. Project Readme & Operational Guide

Source File: `README.md`

Quickstart commands, system requirements, demo user credentials, and walkthrough guide.

```
# Faculty Feedback & Appraisal System

A full-stack academic analytics platform that digitizes semester-end faculty
feedback, automatically computes faculty appraisal scores, and gives the
HOD a real analytics dashboard — built as a college project, structured
like production software.

- **Backend:** Node.js, Express, SQLite (via `better-sqlite3`), JWT auth
- **Frontend:** React (Vite), Tailwind CSS, Recharts, React Router
- See **[ARCHITECTURE.md](./ARCHITECTURE.md)** for schema, API design, appraisal methodology, and the full viva-ready proje
ct writeup.

> **Why SQLite instead of MongoDB?** The brief suggested MongoDB, but a
> normalized relational schema (users → students/faculty → subjects →
> feedback → ratings) is a much more natural fit for this data, and
> SQLite needs zero setup — no server to install or connection string to
> configure. The schema is fully normalized SQL and maps directly onto
> MySQL or PostgreSQL if you'd rather run a client-server database; only
> `backend/config/db.js` would need to change.

---

## 1. Quick Start

### Prerequisites
- Node.js 18+ (tested and compatible with **Node v22** and **Node v24**) and npm

### Quick Start (from root)

```bash
# 1. Seed demo database
npm run seed

# 2. Run backend (terminal 1)
npm run start:backend     # runs on http://localhost:5000

# 3. Run frontend (terminal 2)
npm run start:frontend    # runs on http://localhost:5173
```

Alternatively, you can run them from each directory:

Backend

```bash
cd backend
npm install
cp .env.example .env
npm run seed      # creates & populates the database with demo data
npm start         # runs on http://localhost:5000
```

Frontend (in a second terminal)

```bash
cd frontend
npm install
npm run dev        # runs on http://localhost:5173
```

Open **http://localhost:5173** and sign in with any demo account below.

Demo accounts (password for all: `Password@123`)

Role	Email
HOD/Admin	`hod@college.edu`
Faculty	`faculty1@college.edu`
Student	`student1@college.edu`
```

```
69
70  Run `npm run seed` again at any time to reset the database to a clean demo state.
71
72  ---
73
74  ## 2. Project Structure
75
76  ```
77  faculty-feedback-system/
78  ├── backend/
79  │   ├── config/db.js          # SQLite connection + schema definition
80  │   ├── controllers/          # request handlers (business logic)
81  │   ├── routes/               # Express routers, one per resource
82  │   ├── middleware/auth.js    # JWT auth + role-based authorization
83  │   ├── utils/appraisal.js    # the appraisal scoring engine
84  │   ├── seed.js               # demo data generator
85  │   └── server.js             # app entrypoint
86  └── frontend/
87      └── src/
88          ├── api/client.js      # axios instance with auth interceptor
89          ├── context/AuthContext.jsx
90          ├── components/        # Layout, KPICard, RatingInput, etc.
91          └── pages/
92              ├── Login.jsx
93              ├── student/        # feedback submission flow
94              ├── faculty/        # performance dashboard
95              └── hod/            # analytics, admin, appraisals, reports
96  ```
97
98  ## 3. Running a demo / viva walkthrough
99
100 1. **Student** (`student1@college.edu`) → Give Feedback → rate a subject → submit → see the confirmation screen and the com
pletion progress bar update.
101 2. **Faculty** (`faculty1@college.edu`) → My Performance → see KPI cards, the radar chart of criteria, the semester trend l
ine, rating distribution, and anonymized comments — with the exact appraisal formula shown on screen.
102 3. **HOD** (`hod@college.edu`) → Dashboard → filter by department, see faculty comparison, criterion radar, semester trend,
and recent activity → Appraisals to regenerate/view scores → Reports to export a CSV → Settings to change the appraisal formula th
resholds or open/close the feedback collection window.
103
104 ## 4. Notes for production use
105
106 - Replace `JWT_SECRET` in `backend/.env` with a long random value.
107 - `better-sqlite3` is fine for a class project / single-server demo; for real concurrent multi-department use, swap in Post
greSQL (the SQL is already portable) and move to connection pooling.
108 - Add rate limiting and HTTPS termination (e.g. via a reverse proxy) before any real deployment.
109
```

CHAPTER 2: DATABASE LAYER & INITIALIZATION

DATABASE LAYER

3. Database Configuration & 8-Table Normalized Schema

📁 Source File: `backend/config/db.js`

SQLite connection initialization, foreign key pragma, table schemas (users, students, faculty, subjects, criteria, feedback_periods, feedback, feedback_ratings, appraisal_config, appraisal) and indexing.

```
/**
 * Database configuration.
 *
 * Uses better-sqlite3 for a zero-config, file-based relational database.
 * This keeps the project trivially runnable for a college demo / viva
 * (no external DB server needed) while still using real SQL, foreign keys,
 * transactions and normalized tables — the same schema maps 1:1 onto
 * MySQL/Postgres if you want to swap engines later.
 */
const path = require("path");
const Database = require("better-sqlite3");

const DB_PATH = process.env.DB_PATH || path.join(__dirname, "..", "data", "app.db");

const fs = require("fs");
fs.mkdirSync(path.dirname(DB_PATH), { recursive: true });

const db = new Database(DB_PATH);
db.pragma("journal_mode = WAL");
db.pragma("foreign_keys = ON");

function initSchema() {
  db.exec(`
    CREATE TABLE IF NOT EXISTS users (
      id            INTEGER PRIMARY KEY AUTOINCREMENT,
      name          TEXT NOT NULL,
      email         TEXT NOT NULL UNIQUE,
      password_hash TEXT NOT NULL,
      role          TEXT NOT NULL CHECK (role IN ('student','faculty','hod')),
      department    TEXT NOT NULL,
      created_at    TEXT NOT NULL DEFAULT (datetime('now'))
    );

    CREATE TABLE IF NOT EXISTS students (
      id           INTEGER PRIMARY KEY AUTOINCREMENT,
      user_id      INTEGER NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,
      department   TEXT NOT NULL,
      semester     INTEGER NOT NULL,
      section      TEXT NOT NULL DEFAULT 'A',
      roll_number  TEXT
    );

    CREATE TABLE IF NOT EXISTS faculty (
      id           INTEGER PRIMARY KEY AUTOINCREMENT,
      user_id      INTEGER NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,
      department   TEXT NOT NULL,
      designation  TEXT NOT NULL DEFAULT 'Assistant Professor'
    );

    CREATE TABLE IF NOT EXISTS subjects (
      id            INTEGER PRIMARY KEY AUTOINCREMENT,
      subject_name  TEXT NOT NULL,
      subject_code  TEXT,
      faculty_id    INTEGER NOT NULL REFERENCES faculty(id) ON DELETE CASCADE,
      department    TEXT NOT NULL,
      semester      INTEGER NOT NULL
    );

    CREATE TABLE IF NOT EXISTS criteria (
      id          INTEGER PRIMARY KEY AUTOINCREMENT,
      name        TEXT NOT NULL UNIQUE,
      weight      REAL NOT NULL DEFAULT 1.0,   -- relative weight used in appraisal formula
      is_active   INTEGER NOT NULL DEFAULT 1,
      sort_order  INTEGER NOT NULL DEFAULT 0
    );

    CREATE TABLE IF NOT EXISTS feedback_periods (
      id             INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
      academic_year  TEXT NOT NULL,
      semester       INTEGER NOT NULL,
      is_open        INTEGER NOT NULL DEFAULT 1,
      allow_editing  INTEGER NOT NULL DEFAULT 0,
      opened_at      TEXT NOT NULL DEFAULT (datetime('now')),
      closed_at      TEXT
    );

    CREATE TABLE IF NOT EXISTS feedback (
      id             INTEGER PRIMARY KEY AUTOINCREMENT,
      student_id     INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,
      faculty_id     INTEGER NOT NULL REFERENCES faculty(id) ON DELETE CASCADE,
      subject_id     INTEGER NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
      period_id      INTEGER NOT NULL REFERENCES feedback_periods(id) ON DELETE CASCADE,
      semester       INTEGER NOT NULL,
      academic_year  TEXT NOT NULL,
      comments       TEXT,
      submitted_at   TEXT NOT NULL DEFAULT (datetime('now')),
      UNIQUE(student_id, faculty_id, subject_id, period_id)
    );

    CREATE TABLE IF NOT EXISTS feedback_ratings (
      id           INTEGER PRIMARY KEY AUTOINCREMENT,
      feedback_id  INTEGER NOT NULL REFERENCES feedback(id) ON DELETE CASCADE,
      criterion_id INTEGER NOT NULL REFERENCES criteria(id) ON DELETE CASCADE,
      rating       INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),
      UNIQUE(feedback_id, criterion_id)
    );

    CREATE TABLE IF NOT EXISTS appraisal_config (
      id                 INTEGER PRIMARY KEY AUTOINCREMENT,
      scale_max          REAL NOT NULL DEFAULT 5,
      pass_threshold      REAL NOT NULL DEFAULT 3.0,   -- min avg rating considered "satisfactory"
      excellent_threshold REAL NOT NULL DEFAULT 4.5,
      formula_notes       TEXT NOT NULL DEFAULT 'appraisal_percent = (weighted_average_rating / scale_max) * 100'
    );

    CREATE TABLE IF NOT EXISTS appraisal (
      id               INTEGER PRIMARY KEY AUTOINCREMENT,
      faculty_id       INTEGER NOT NULL REFERENCES faculty(id) ON DELETE CASCADE,
      semester         INTEGER NOT NULL,
      academic_year    TEXT NOT NULL,
      total_responses  INTEGER NOT NULL,
      average_rating   REAL NOT NULL,
      appraisal_score  REAL NOT NULL,
      grade            TEXT NOT NULL,
      generated_at     TEXT NOT NULL DEFAULT (datetime('now')),
      UNIQUE(faculty_id, semester, academic_year)
    );

    CREATE INDEX IF NOT EXISTS idx_feedback_faculty ON feedback(faculty_id);
    CREATE INDEX IF NOT EXISTS idx_feedback_student ON feedback(student_id);
    CREATE INDEX IF NOT EXISTS idx_subjects_faculty ON subjects(faculty_id);
  `);

  const cfgCount = db.prepare("SELECT COUNT(*) c FROM appraisal_config").get().c;
  if (cfgCount === 0) {
    db.prepare("INSERT INTO appraisal_config DEFAULT VALUES").run();
  }
}

initSchema();

module.exports = db;

```

CHAPTER 3: CORE APPRAISAL ENGINE & SECURITY MIDDLEWARE

BUSINESS LOGIC

4. Appraisal Scoring & Mathematical Grade Engine

📁 Source File: `backend/utils/appraisal.js`

Mathematical scoring engine calculating criterion-wise averages, weighted ratings, 0-100 normalized performance percentages, and grade thresholds.

```
/**
 * Appraisal Calculation Engine
 * ----------------------------
 * Transparent, configurable methodology:
 *
 * 1. For each criterion, compute the criterion average rating across all
 *    feedback submitted for a faculty member in a given semester/year:
 *
 *        criterionAvg = sum(ratings for criterion) / count(ratings for criterion)
 *
 * 2. Compute the WEIGHTED average rating across all criteria, using each
 *    criterion's configurable weight (defaults to 1.0 = equal weighting):
 *
 *        weightedAverage = sum(criterionAvg_i * weight_i) / sum(weight_i)
 *
 * 3. Normalize against the rating scale (default 1-5) into a 0-100 score:
 *
 *        appraisalScore = (weightedAverage / scaleMax) * 100
 *
 * 4. Assign a letter grade using configurable thresholds:
 *        >= excellentThreshold        -> "Excellent"
 *        >= passThreshold             -> "Satisfactory"
 *        <  passThreshold             -> "Needs Improvement"
 *
 * This whole method is intentionally simple linear math (no black box /
 * ML) so it can be explained in a viva and audited by the HOD.
 */
const db = require("../config/db");

function getConfig() {
  return db.prepare("SELECT * FROM appraisal_config ORDER BY id DESC LIMIT 1").get();
}

function computeCriterionAverages(facultyId, semester, academicYear) {
  // The feedback join carries the faculty/semester/year filter in its ON clause
  // (not WHERE), so every criterion row survives the LEFT JOIN even with zero
  // matches. We then use CASE WHEN f.id IS NOT NULL to only average/count the
  // ratings that actually matched — otherwise a LEFT JOIN silently counts every
  // rating for that criterion across ALL faculty and periods.
  const rows = db
    .prepare(
      `SELECT c.id AS criterion_id, c.name, c.weight,
              AVG(CASE WHEN f.id IS NOT NULL THEN fr.rating END) AS avg_rating,
              COUNT(CASE WHEN f.id IS NOT NULL THEN fr.rating END) AS rating_count
       FROM criteria c
       LEFT JOIN feedback_ratings fr ON fr.criterion_id = c.id
       LEFT JOIN feedback f ON f.id = fr.feedback_id
         AND f.faculty_id = ? AND f.semester = ? AND f.academic_year = ?
       WHERE c.is_active = 1
       GROUP BY c.id
       ORDER BY c.sort_order ASC, c.id ASC`
    )
    .all(facultyId, semester, academicYear);

  return rows.map((r) => ({
    criterionId: r.criterion_id,
    name: r.name,
    weight: r.weight,
    averageRating: r.avg_rating ? Number(r.avg_rating.toFixed(2)) : 0,
    ratingCount: r.rating_count || 0,
  }));
}

function gradeFor(weightedAverage, config) {
  if (weightedAverage >= config.excellent_threshold) return "Excellent";
  if (weightedAverage >= config.pass_threshold) return "Satisfactory";
  return "Needs Improvement";
}
```

```
69
70  /**
71   * Computes and upserts the appraisal record for a faculty/semester/year.
72   * Returns the full breakdown for transparency.
73   */
74  function generateAppraisal(facultyId, semester, academicYear) {
75    const config = getConfig();
76    const criterionAverages = computeCriterionAverages(facultyId, semester, academicYear);
77
78    const totalResponses = db
79      .prepare(
80        `SELECT COUNT(*) c FROM feedback WHERE faculty_id = ? AND semester = ? AND academic_year = ?`
81      )
82      .get(facultyId, semester, academicYear).c;
83
84    const scored = criterionAverages.filter((c) => c.ratingCount > 0);
85    const weightSum = scored.reduce((s, c) => s + c.weight, 0);
86    const weightedAverage =
87      weightSum > 0
88        ? scored.reduce((s, c) => s + c.averageRating * c.weight, 0) / weightSum
89        : 0;
90
91    const appraisalScore = Number(((weightedAverage / config.scale_max) * 100).toFixed(2));
92    const grade = gradeFor(weightedAverage, config);
93
94    if (totalResponses > 0) {
95      db.prepare(
96        `INSERT INTO appraisal (faculty_id, semester, academic_year, total_responses, average_rating, appraisal_score, grade)
97         VALUES (?, ?, ?, ?, ?, ?, ?)
98         ON CONFLICT(faculty_id, semester, academic_year)
99         DO UPDATE SET total_responses = excluded.total_responses,
100                      average_rating = excluded.average_rating,
101                      appraisal_score = excluded.appraisal_score,
102                      grade = excluded.grade,
103                      generated_at = datetime('now')`
104      ).run(facultyId, semester, academicYear, totalResponses, Number(weightedAverage.toFixed(2)), appraisalScore, grade);
105    }
106
107    return {
108      facultyId,
109      semester,
110      academicYear,
111      totalResponses,
112      criterionAverages,
113      weightedAverageRating: Number(weightedAverage.toFixed(2)),
114      scaleMax: config.scale_max,
115      appraisalScore,
116      grade,
117      methodology: {
118        step1: "criterionAvg = sum(ratings for criterion) / count(ratings for criterion)",
119        step2: "weightedAverage = sum(criterionAvg_i * weight_i) / sum(weight_i)",
120        step3: `appraisalScore = (weightedAverage / ${config.scale_max}) * 100`,
121        thresholds: {
122          excellent: config.excellent_threshold,
123          satisfactory: config.pass_threshold,
124        },
125      },
126    };
127  }
128
129  module.exports = { generateAppraisal, computeCriterionAverages, getConfig };
130
```

CHAPTER 3: CORE APPRAISAL ENGINE & SECURITY MIDDLEWARE

SECURITY / RBAC

5. JWT Authentication & Role Authorization Middleware

📁 Source File: `backend/middleware/auth.js`

JSON Web Token verification, session integrity, and role-based route gating (Student, Faculty, HOD).

```
const jwt = require("jsonwebtoken");

const JWT_SECRET = process.env.JWT_SECRET || "dev-secret-change-me";

function authenticate(req, res, next) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null;

  if (!token) {
    return res.status(401).json({ success: false, message: "Authentication token missing. Please log in again." });
  }

  try {
    const payload = jwt.verify(token, JWT_SECRET);
    req.user = payload; // { id, role, email, name, department, profileId }
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: "Session expired or invalid. Please log in again." });
  }
}

function authorize(...roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ success: false, message: "You do not have permission to access this resource." });
    }
    next();
  };
}

module.exports = { authenticate, authorize, JWT_SECRET };

```

CHAPTER 3: CORE APPRAISAL ENGINE & SECURITY MIDDLEWARE

SERVER CORE

6. Express Server Entrypoint & API Dispatcher

📁 Source File: `backend/server.js`

Express server bootstrap, CORS configuration, JSON body parsing, route mounts, and global error handling.

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const morgan = require("morgan");

const authRoutes = require("./routes/authRoutes");
const feedbackRoutes = require("./routes/feedbackRoutes");
const facultyRoutes = require("./routes/facultyRoutes");
const analyticsRoutes = require("./routes/analyticsRoutes");
const appraisalRoutes = require("./routes/appraisalRoutes");
const adminRoutes = require("./routes/adminRoutes");

const app = express();

app.use(cors());
app.use(express.json());
app.use(morgan("dev"));

app.get("/api/health", (req, res) => res.json({ success: true, message: "API is running." }));

app.use("/api/auth", authRoutes);
app.use("/api/feedback", feedbackRoutes);
app.use("/api/faculty", facultyRoutes);
app.use("/api/analytics", analyticsRoutes);
app.use("/api/appraisal", appraisalRoutes);
app.use("/api/admin", adminRoutes);

// 404 handler
app.use((req, res) => {
  res.status(404).json({ success: false, message: "Route not found." });
});

// Centralized error handler
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({
    success: false,
    message: err.message || "Internal server error. Please try again later.",
  });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Faculty Feedback & Appraisal System API running on http://localhost:${PORT}`);
});

```

CHAPTER 4: BACKEND CONTROLLERS (BUSINESS LOGIC)

CONTROLLER

7. Authentication & Student Self-Registration Controller

📁 Source File: `backend/controllers/authController.js`

User login authentication, bcrypt password matching, JWT issuance, and student registration.

```
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const db = require("../config/db");
const { JWT_SECRET } = require("../middleware/auth");

function signToken(user, profileId) {
  return jwt.sign(
    {
      id: user.id,
      name: user.name,
      email: user.email,
      role: user.role,
      department: user.department,
      profileId, // students.id or faculty.id, null for HOD
    },
    JWT_SECRET,
    { expiresIn: "8h" }
  );
}

exports.login = (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ success: false, message: "Email and password are required." });
  }

  const user = db.prepare("SELECT * FROM users WHERE email = ?").get(email.toLowerCase().trim());
  if (!user) {
    return res.status(401).json({ success: false, message: "Invalid email or password." });
  }

  const valid = bcrypt.compareSync(password, user.password_hash);
  if (!valid) {
    return res.status(401).json({ success: false, message: "Invalid email or password." });
  }

  let profileId = null;
  if (user.role === "student") {
    profileId = db.prepare("SELECT id FROM students WHERE user_id = ?").get(user.id)?.id;
  } else if (user.role === "faculty") {
    profileId = db.prepare("SELECT id FROM faculty WHERE user_id = ?").get(user.id)?.id;
  }

  const token = signToken(user, profileId);
  res.json({
    success: true,
    data: {
      token,
      user: {
        id: user.id,
        name: user.name,
        email: user.email,
        role: user.role,
        department: user.department,
        profileId,
      },
    },
  });
};

exports.register = (req, res) => {
  const { name, email, password, department, semester, section, rollNumber } = req.body;
  if (!name || !email || !password || !department || !semester) {
    return res.status(400).json({ success: false, message: "Name, email, password, department and semester are required."
});
  }

  const cleanEmail = email.toLowerCase().trim();
  const exists = db.prepare("SELECT id FROM users WHERE email = ?").get(cleanEmail);
```

```
  if (exists) {
    return res.status(409).json({ success: false, message: "A user with this email already exists." });
  }

  const tx = db.transaction(() => {
    const hash = bcrypt.hashSync(password, 10);
    const userRes = db
      .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'student', ?)")
      .run(name.trim(), cleanEmail, hash, department.trim());
    const stuRes = db
      .prepare("INSERT INTO students (user_id, department, semester, section, roll_number) VALUES (?, ?, ?, ?, ?)")
      .run(userRes.lastInsertRowid, department.trim(), Number(semester), section || "A", rollNumber ? rollNumber.trim() : n
ull);

    return {
      userId: userRes.lastInsertRowid,
      studentId: stuRes.lastInsertRowid,
    };
  });

  const { userId, studentId } = tx();
  const user = db.prepare("SELECT * FROM users WHERE id = ?").get(userId);
  const token = signToken(user, studentId);

  res.status(201).json({
    success: true,
    message: "Registration successful!",
    data: {
      token,
      user: {
        id: user.id,
        name: user.name,
        email: user.email,
        role: user.role,
        department: user.department,
        profileId: studentId,
      },
    },
  });
};

exports.me = (req, res) => {
  res.json({ success: true, data: req.user });
};


```

CHAPTER 4: BACKEND CONTROLLERS (BUSINESS LOGIC)

CONTROLLER

8. Feedback Submission & Validation Controller

📁 Source File: `backend/controllers/feedbackController.js`

Student feedback validation, multi-criterion ratings transactions, duplicate submission guard, and progress tracking.

```
const db = require("../config/db");

/** GET /api/feedback/targets — faculty/subjects this student can review, with status */
exports.getTargets = (req, res) => {
  const studentId = req.user.profileId;
  const student = db.prepare("SELECT * FROM students WHERE id = ?").get(studentId);
  if (!student) return res.status(404).json({ success: false, message: "Student profile not found." });

  const period = db
    .prepare("SELECT * FROM feedback_periods WHERE is_open = 1 ORDER BY id DESC LIMIT 1")
    .get();

  const subjects = db
    .prepare(
      `SELECT s.id AS subject_id, s.subject_name, s.subject_code,
              f.id AS faculty_id, u.name AS faculty_name, f.designation
       FROM subjects s
       JOIN faculty f ON f.id = s.faculty_id
       JOIN users u ON u.id = f.user_id
       WHERE s.department = ? AND s.semester = ?
       ORDER BY s.subject_name`
    )
    .all(student.department, student.semester);

  const submitted = period
    ? new Set(
        db
          .prepare(
            `SELECT faculty_id || '-' || subject_id AS k FROM feedback
             WHERE student_id = ? AND period_id = ?`
          )
          .all(studentId, period.id)
          .map((r) => r.k)
      )
    : new Set();

  const targets = subjects.map((s) => ({
    ...s,
    submitted: submitted.has(`${s.faculty_id}-${s.subject_id}`),
  }));

  res.json({
    success: true,
    data: {
      periodOpen: !!period,
      period,
      allowEditing: period ? !!period.allow_editing : false,
      targets,
    },
  });
};

/** GET /api/criteria — active evaluation criteria for the feedback form */
exports.getCriteria = (req, res) => {
  const criteria = db
    .prepare("SELECT id, name, weight FROM criteria WHERE is_active = 1 ORDER BY sort_order ASC, id ASC")
    .all();
  res.json({ success: true, data: criteria });
};

/** POST /api/feedback — submit feedback for one faculty/subject */
exports.submitFeedback = (req, res) => {
  const studentId = req.user.profileId;
  const { facultyId, subjectId, ratings, comments } = req.body;

  if (!facultyId || !subjectId || !Array.isArray(ratings) || ratings.length === 0) {
    return res.status(400).json({ success: false, message: "facultyId, subjectId and ratings are required." });
  }
  for (const r of ratings) {
```

```
    70      if (!r.criterionId || !Number.isInteger(r.rating) || r.rating < 1 || r.rating > 5) {
    71        return res.status(400).json({ success: false, message: "Each rating must include criterionId and an integer rating fr
om 1 to 5." });
    72      }
    73    }
    74
    75    const period = db.prepare("SELECT * FROM feedback_periods WHERE is_open = 1 ORDER BY id DESC LIMIT 1").get();
    76    if (!period) {
    77      return res.status(400).json({ success: false, message: "Feedback collection is currently closed." });
    78    }
    79
    80    const subject = db.prepare("SELECT * FROM subjects WHERE id = ?").get(subjectId);
    81    if (!subject || subject.faculty_id !== facultyId) {
    82      return res.status(400).json({ success: false, message: "Invalid faculty/subject combination." });
    83    }
    84
    85    const existing = db
    86      .prepare(
    87        `SELECT id FROM feedback WHERE student_id = ? AND faculty_id = ? AND subject_id = ? AND period_id = ?`
    88      )
    89      .get(studentId, facultyId, subjectId, period.id);
    90
    91    if (existing && !period.allow_editing) {
    92      return res.status(409).json({ success: false, message: "You have already submitted feedback for this faculty member."
});
    93    }
    94
    95    const tx = db.transaction(() => {
    96      let feedbackId;
    97      if (existing) {
    98        db.prepare("UPDATE feedback SET comments = ?, submitted_at = datetime('now') WHERE id = ?").run(
    99          comments || null,
   100          existing.id
   101        );
   102        feedbackId = existing.id;
   103        db.prepare("DELETE FROM feedback_ratings WHERE feedback_id = ?").run(feedbackId);
   104      } else {
   105        const info = db
   106          .prepare(
   107            `INSERT INTO feedback (student_id, faculty_id, subject_id, period_id, semester, academic_year, comments)
   108             VALUES (?, ?, ?, ?, ?, ?, ?)`
   109          )
   110          .run(studentId, facultyId, subjectId, period.id, period.semester, period.academic_year, comments || null);
   111        feedbackId = info.lastInsertRowid;
   112      }
   113
   114      const insertRating = db.prepare(
   115        "INSERT INTO feedback_ratings (feedback_id, criterion_id, rating) VALUES (?, ?, ?)"
   116      );
   117      for (const r of ratings) insertRating.run(feedbackId, r.criterionId, r.rating);
   118
   119      return feedbackId;
   120    });
   121
   122    try {
   123      const feedbackId = tx();
   124      res.status(201).json({
   125        success: true,
   126        message: "Feedback submitted successfully. Thank you for contributing to academic improvement.",
   127        data: { feedbackId },
   128      });
   129    } catch (err) {
   130      res.status(500).json({ success: false, message: "Could not save feedback due to a server error." });
   131    }
   132  };
   133
   134  /** GET /api/feedback/status — overall completion status for the logged-in student */
   135  exports.getStatus = (req, res) => {
   136    const studentId = req.user.profileId;
   137    const student = db.prepare("SELECT * FROM students WHERE id = ?").get(studentId);
   138    const totalSubjects = db
   139      .prepare("SELECT COUNT(*) c FROM subjects WHERE department = ? AND semester = ?")
   140      .get(student.department, student.semester).c;
   141
   142    const period = db.prepare("SELECT * FROM feedback_periods WHERE is_open = 1 ORDER BY id DESC LIMIT 1").get();
   143    const completed = period
   144      ? db.prepare("SELECT COUNT(*) c FROM feedback WHERE student_id = ? AND period_id = ?").get(studentId, period.id).c
   145      : 0;
   146
   147    res.json({
```

```
    success: true,
    data: {
      totalSubjects,
      completed,
      percentage: totalSubjects ? Math.round((completed / totalSubjects) * 100) : 0,
      periodOpen: !!period,
    },
  });
};

```

CHAPTER 4: BACKEND CONTROLLERS (BUSINESS LOGIC)

CONTROLLER

9. Analytics & KPI Aggregation Controller

📁 Source File: `backend/controllers/analyticsController.js`

Real-time analytics computation, department completion rates, ratings distribution, and radar chart metrics.

```
const db = require("../config/db");

function currentPeriod() {
  return db.prepare("SELECT * FROM feedback_periods WHERE is_open = 1 ORDER BY id DESC LIMIT 1").get()
    || db.prepare("SELECT * FROM feedback_periods ORDER BY is_open DESC, id DESC LIMIT 1").get();
}

/** GET /api/analytics/dashboard — HOD overview KPIs */
exports.getDashboard = (req, res) => {
  const { department, semester, academicYear } = req.query;
  const period = currentPeriod();
  const sem = semester || period?.semester;
  const year = academicYear || period?.academic_year;

  const deptFilter = department ? "AND u.department = ?" : "";
  const deptParam = department ? [department] : [];

  const totalStudents = db
    .prepare(`SELECT COUNT(*) c FROM students s JOIN users u ON u.id = s.user_id WHERE 1=1 ${deptFilter}`)
    .get(...deptParam).c;

  const totalFaculty = db
    .prepare(`SELECT COUNT(*) c FROM faculty f JOIN users u ON u.id = f.user_id WHERE 1=1 ${deptFilter}`)
    .get(...deptParam).c;

  const fbDeptFilter = department ? "AND f.department = ?" : "";
  const totalFeedback = db
    .prepare(
      `SELECT COUNT(*) c FROM feedback fb JOIN faculty f ON f.id = fb.faculty_id
       WHERE fb.semester = ? AND fb.academic_year = ? ${fbDeptFilter}`
    )
    .get(sem, year, ...deptParam).c;

  const expectedResponses = db
    .prepare(
      `SELECT COUNT(*) c FROM students s JOIN users u ON u.id = s.user_id
       JOIN subjects sub ON sub.department = s.department AND sub.semester = s.semester
       WHERE s.semester = ? ${deptFilter}`
    )
    .get(sem, ...deptParam).c;

  const avgRatingRow = db
    .prepare(
      `SELECT AVG(fr.rating) a FROM feedback_ratings fr
       JOIN feedback fb ON fb.id = fr.feedback_id
       JOIN faculty f ON f.id = fb.faculty_id
       WHERE fb.semester = ? AND fb.academic_year = ? ${fbDeptFilter}`
    )
    .get(sem, year, ...deptParam);

  res.json({
    success: true,
    data: {
      semester: sem,
      academicYear: year,
      totalStudents,
      totalFaculty,
      totalFeedback,
      completionPercentage: expectedResponses ? Math.round((totalFeedback / expectedResponses) * 100) : 0,
      departmentAverageRating: avgRatingRow.a ? Number(avgRatingRow.a.toFixed(2)) : 0,
    },
  });
};

/** GET /api/analytics/faculty — faculty-wise average rating (for bar chart / comparison) */
exports.getFacultyComparison = (req, res) => {
  const { department, semester, academicYear } = req.query;
  const period = currentPeriod();
  const sem = semester || period?.semester;
```

```
  const year = academicYear || period?.academic_year;
  const deptFilter = department ? "AND f.department = ?" : "";
  const deptParam = department ? [department] : [];

  const rows = db
    .prepare(
      `SELECT f.id AS faculty_id, u.name, f.department,
              AVG(fr.rating) AS avg_rating, COUNT(DISTINCT fb.id) AS responses
       FROM faculty f
       JOIN users u ON u.id = f.user_id
       LEFT JOIN feedback fb ON fb.faculty_id = f.id AND fb.semester = ? AND fb.academic_year = ?
       LEFT JOIN feedback_ratings fr ON fr.feedback_id = fb.id
       WHERE 1=1 ${deptFilter}
       GROUP BY f.id
       ORDER BY avg_rating DESC`
    )
    .all(sem, year, ...deptParam)
    .map((r) => ({
      facultyId: r.faculty_id,
      name: r.name,
      department: r.department,
      averageRating: r.avg_rating ? Number(r.avg_rating.toFixed(2)) : 0,
      responses: r.responses,
    }));

  res.json({ success: true, data: rows });
};

/** GET /api/analytics/criteria — criterion-wise average across department (radar chart) */
exports.getCriteriaBreakdown = (req, res) => {
  const { department, semester, academicYear, facultyId } = req.query;
  const period = currentPeriod();
  const sem = semester || period?.semester;
  const year = academicYear || period?.academic_year;

  let fbFilter = "fb.semester = ? AND fb.academic_year = ?";
  const params = [sem, year];
  let facultyFilter = "1=1";
  if (department) {
    facultyFilter += " AND f.department = ?";
    params.push(department);
  }
  if (facultyId) {
    facultyFilter += " AND f.id = ?";
    params.push(facultyId);
  }

  // Filters are applied inside the LEFT JOIN conditions (not WHERE) so that
  // criteria with zero matching ratings still appear in the result with avg_rating = 0,
  // instead of being dropped from the radar chart entirely.
  const rows = db
    .prepare(
      `SELECT c.name,
              AVG(CASE WHEN f.id IS NOT NULL THEN fr.rating END) avg_rating
       FROM criteria c
       LEFT JOIN feedback_ratings fr ON fr.criterion_id = c.id
       LEFT JOIN feedback fb ON fb.id = fr.feedback_id AND ${fbFilter}
       LEFT JOIN faculty f ON f.id = fb.faculty_id AND ${facultyFilter}
       WHERE c.is_active = 1
       GROUP BY c.id ORDER BY c.sort_order`
    )
    .all(sem, year, ...(department ? [department] : []), ...(facultyId ? [facultyId] : []));

  res.json({
    success: true,
    data: rows.map((r) => ({ criterion: r.name, averageRating: r.avg_rating ? Number(r.avg_rating.toFixed(2)) : 0 })),
  });
};

/** GET /api/analytics/trend — semester-wise trend of department average rating */
exports.getSemesterTrend = (req, res) => {
  const { department } = req.query;
  const deptFilter = department ? "WHERE f.department = ?" : "";
  const deptParam = department ? [department] : [];

  const rows = db
    .prepare(
      `SELECT a.academic_year, a.semester, AVG(a.average_rating) avg_rating, SUM(a.total_responses) responses
       FROM appraisal a JOIN faculty f ON f.id = a.faculty_id
       ${deptFilter}
```

```
        GROUP BY a.academic_year, a.semester
        ORDER BY a.academic_year, a.semester`
      )
      .all(...deptParam);

    res.json({
      success: true,
      data: rows.map((r) => ({
        label: `${r.academic_year} · Sem ${r.semester}`,
        averageRating: r.avg_rating ? Number(r.avg_rating.toFixed(2)) : 0,
        responses: r.responses,
      })),
    });
  };

  /** GET /api/analytics/recent-activity */
  exports.getRecentActivity = (req, res) => {
    const rows = db
      .prepare(
        `SELECT fb.submitted_at, u.name AS faculty_name, s.subject_name
         FROM feedback fb
         JOIN faculty f ON f.id = fb.faculty_id
         JOIN users u ON u.id = f.user_id
         JOIN subjects s ON s.id = fb.subject_id
         ORDER BY fb.submitted_at DESC LIMIT 15`
      )
      .all();
    res.json({ success: true, data: rows });
  };

  /** GET /api/analytics/filters — distinct departments/semesters/faculty/subjects for filter dropdowns */
  exports.getFilterOptions = (req, res) => {
    const departments = db.prepare("SELECT DISTINCT department FROM users ORDER BY department").all().map((r) => r.departmen
t);
    const semesters = db.prepare("SELECT DISTINCT semester FROM students ORDER BY semester").all().map((r) => r.semester);
    const academicYears = db.prepare("SELECT DISTINCT academic_year FROM feedback_periods ORDER BY academic_year").all().map
((r) => r.academic_year);
    const faculty = db
      .prepare("SELECT f.id, u.name FROM faculty f JOIN users u ON u.id = f.user_id ORDER BY u.name")
      .all();
    const criteria = db.prepare("SELECT id, name FROM criteria WHERE is_active = 1 ORDER BY sort_order").all();
    res.json({ success: true, data: { departments, semesters, academicYears, faculty, criteria } });
  };

```

CHAPTER 4: BACKEND CONTROLLERS (BUSINESS LOGIC)

CONTROLLER

10. Faculty Dashboard Performance Controller

Source File: `backend/controllers/facultyController.js`

Per-faculty performance summaries, semester appraisal retrieval, criteria breakdowns, and anonymous student comments.

```
const db = require("../config/db");
const { generateAppraisal } = require("../utils/appraisal");

exports.listFaculty = (req, res) => {
  const { department, semester } = req.query;
  let sql = `
    SELECT f.id, u.name, u.email, f.department, f.designation
    FROM faculty f JOIN users u ON u.id = f.user_id WHERE 1=1`;
  const params = [];
  if (department) {
    sql += " AND f.department = ?";
    params.push(department);
  }
  sql += " ORDER BY u.name";
  const rows = db.prepare(sql).all(...params);
  res.json({ success: true, data: rows });
};

exports.getFacultyById = (req, res) => {
  const row = db
    .prepare(
      `SELECT f.id, u.name, u.email, f.department, f.designation
       FROM faculty f JOIN users u ON u.id = f.user_id WHERE f.id = ?`
    )
    .get(req.params.id);
  if (!row) return res.status(404).json({ success: false, message: "Faculty not found." });

  const subjects = db
    .prepare("SELECT id, subject_name, subject_code, semester FROM subjects WHERE faculty_id = ?")
    .all(req.params.id);

  res.json({ success: true, data: { ...row, subjects } });
};

/** GET /api/faculty/:id/performance?semester=&academicYear= */
exports.getPerformance = (req, res) => {
  const facultyId = Number(req.params.id);

  // Faculty may only view their own performance
  if (req.user.role === "faculty" && req.user.profileId !== facultyId) {
    return res.status(403).json({ success: false, message: "You can only view your own performance data." });
  }

  const facultyRow = db
    .prepare(`SELECT f.id, u.name, f.department, f.designation FROM faculty f JOIN users u ON u.id = f.user_id WHERE f.id =
?`)
    .get(facultyId);
  if (!facultyRow) return res.status(404).json({ success: false, message: "Faculty not found." });

  const latestPeriod = db.prepare("SELECT * FROM feedback_periods ORDER BY is_open DESC, id DESC LIMIT 1").get();
  const semester = Number(req.query.semester) || latestPeriod?.semester;
  const academicYear = req.query.academicYear || latestPeriod?.academic_year;

  const appraisal = semester && academicYear ? generateAppraisal(facultyId, semester, academicYear) : null;

  // Semester-wise trend (all historical appraisal rows for this faculty)
  const trend = db
    .prepare(
      `SELECT semester, academic_year, average_rating, appraisal_score, total_responses
       FROM appraisal WHERE faculty_id = ? ORDER BY academic_year, semester`
    )
    .all(facultyId);

  // Rating distribution (count of each 1-5 rating across all criteria, current period)
  const distribution = db
    .prepare(
      `SELECT fr.rating, COUNT(*) c
       FROM feedback_ratings fr JOIN feedback f ON f.id = fr.feedback_id
       WHERE f.faculty_id = ? AND f.semester = ? AND f.academic_year = ?
```

```
        GROUP BY fr.rating ORDER BY fr.rating`
    )
    .all(facultyId, semester, academicYear);

  // Aggregated (anonymous) comments — no student identity attached
  const comments = db
    .prepare(
      `SELECT comments FROM feedback
       WHERE faculty_id = ? AND semester = ? AND academic_year = ? AND comments IS NOT NULL AND TRIM(comments) != ''
       ORDER BY submitted_at DESC LIMIT 50`
    )
    .all(facultyId, semester, academicYear)
    .map((r) => r.comments);

  res.json({
    success: true,
    data: {
      faculty: facultyRow,
      semester,
      academicYear,
      appraisal,
      trend,
      ratingDistribution: distribution,
      comments,
    },
  });
};

```

CHAPTER 4: BACKEND CONTROLLERS (BUSINESS LOGIC)

CONTROLLER

11. Appraisal Management Controller

📁 Source File: `backend/controllers/appraisalController.js`

HOD appraisal calculation triggers, faculty ranking reports, and configuration controls.

```
const db = require("../config/db");
const { generateAppraisal, getConfig } = require("../utils/appraisal");

/** GET /api/appraisal/:facultyId?semester=&academicYear= */
exports.getAppraisal = (req, res) => {
  const facultyId = Number(req.params.facultyId);
  if (req.user.role === "faculty" && req.user.profileId !== facultyId) {
    return res.status(403).json({ success: false, message: "You can only view your own appraisal." });
  }
  const period = db.prepare("SELECT * FROM feedback_periods ORDER BY is_open DESC, id DESC LIMIT 1").get();
  const semester = Number(req.query.semester) || period?.semester;
  const academicYear = req.query.academicYear || period?.academic_year;

  const result = generateAppraisal(facultyId, semester, academicYear);
  res.json({ success: true, data: result });
};

/** POST /api/appraisal/generate  { facultyId?, semester, academicYear } — HOD/Admin only.
 *  If facultyId omitted, regenerates for ALL faculty. */
exports.generateForFacultyOrAll = (req, res) => {
  const { facultyId, semester, academicYear } = req.body;
  if (!semester || !academicYear) {
    return res.status(400).json({ success: false, message: "semester and academicYear are required." });
  }

  if (facultyId) {
    const result = generateAppraisal(facultyId, semester, academicYear);
    return res.json({ success: true, data: [result] });
  }

  const allFaculty = db.prepare("SELECT id FROM faculty").all();
  const results = allFaculty.map((f) => generateAppraisal(f.id, semester, academicYear));
  res.json({ success: true, message: `Appraisal generated for ${results.length} faculty member(s).`, data: results });
};

/** GET /api/appraisal-config — view current formula configuration */
exports.getAppraisalConfig = (req, res) => {
  res.json({ success: true, data: getConfig() });
};

/** PUT /api/appraisal-config — HOD/Admin updates the formula thresholds */
exports.updateAppraisalConfig = (req, res) => {
  const { scaleMax, passThreshold, excellentThreshold } = req.body;
  const current = getConfig();
  db.prepare(
    `UPDATE appraisal_config SET scale_max = ?, pass_threshold = ?, excellent_threshold = ? WHERE id = ?`
  ).run(
    scaleMax ?? current.scale_max,
    passThreshold ?? current.pass_threshold,
    excellentThreshold ?? current.excellent_threshold,
    current.id
  );
  res.json({ success: true, data: getConfig() });
};

```

CHAPTER 4: BACKEND CONTROLLERS (BUSINESS LOGIC)

CONTROLLER

12. System Administration Controller

📁 Source File: `backend/controllers/adminController.js`

CRUD operations for faculty accounts, student rosters, subject-teacher assignments, criteria weights, and feedback period toggles.

```
const bcrypt = require("bcryptjs");
const db = require("../config/db");

/* ---------- Faculty management ---------- */
exports.createFaculty = (req, res) => {
  const { name, email, password, department, designation } = req.body;
  if (!name || !email || !password || !department) {
    return res.status(400).json({ success: false, message: "name, email, password and department are required." });
  }
  const exists = db.prepare("SELECT id FROM users WHERE email = ?").get(email.toLowerCase());
  if (exists) return res.status(409).json({ success: false, message: "A user with this email already exists." });

  const tx = db.transaction(() => {
    const hash = bcrypt.hashSync(password, 10);
    const userInfo = db
      .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'faculty', ?)")
      .run(name, email.toLowerCase(), hash, department);
    const facInfo = db
      .prepare("INSERT INTO faculty (user_id, department, designation) VALUES (?, ?, ?)")
      .run(userInfo.lastInsertRowid, department, designation || "Assistant Professor");
    return facInfo.lastInsertRowid;
  });

  const facultyId = tx();
  res.status(201).json({ success: true, message: "Faculty member added.", data: { facultyId } });
};

exports.deleteFaculty = (req, res) => {
  const faculty = db.prepare("SELECT * FROM faculty WHERE id = ?").get(req.params.id);
  if (!faculty) return res.status(404).json({ success: false, message: "Faculty not found." });
  db.prepare("DELETE FROM users WHERE id = ?").run(faculty.user_id); // cascades
  res.json({ success: true, message: "Faculty member removed." });
};

/* ---------- Student management ---------- */
exports.createStudent = (req, res) => {
  const { name, email, password, department, semester, section, rollNumber } = req.body;
  if (!name || !email || !password || !department || !semester) {
    return res.status(400).json({ success: false, message: "name, email, password, department and semester are required."
});
  }
  const exists = db.prepare("SELECT id FROM users WHERE email = ?").get(email.toLowerCase());
  if (exists) return res.status(409).json({ success: false, message: "A user with this email already exists." });

  const tx = db.transaction(() => {
    const hash = bcrypt.hashSync(password, 10);
    const userInfo = db
      .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'student', ?)")
      .run(name, email.toLowerCase(), hash, department);
    const stuInfo = db
      .prepare("INSERT INTO students (user_id, department, semester, section, roll_number) VALUES (?, ?, ?, ?, ?)")
      .run(userInfo.lastInsertRowid, department, semester, section || "A", rollNumber || null);
    return stuInfo.lastInsertRowid;
  });

  const studentId = tx();
  res.status(201).json({ success: true, message: "Student added.", data: { studentId } });
};

exports.deleteStudent = (req, res) => {
  const student = db.prepare("SELECT * FROM students WHERE id = ?").get(req.params.id);
  if (!student) return res.status(404).json({ success: false, message: "Student not found." });
  db.prepare("DELETE FROM users WHERE id = ?").run(student.user_id);
  res.json({ success: true, message: "Student removed." });
};

exports.listStudents = (req, res) => {
  const rows = db
    .prepare(
```

```
      `SELECT s.id, u.name, u.email, s.department, s.semester, s.section, s.roll_number
       FROM students s JOIN users u ON u.id = s.user_id ORDER BY u.name`
    )
    .all();
  res.json({ success: true, data: rows });
};

/* ---------- Subject management (assign faculty to subjects) ---------- */
exports.createSubject = (req, res) => {
  const { subjectName, subjectCode, facultyId, department, semester } = req.body;
  if (!subjectName || !facultyId || !department || !semester) {
    return res.status(400).json({ success: false, message: "subjectName, facultyId, department and semester are required."
});
  }
  const info = db
    .prepare("INSERT INTO subjects (subject_name, subject_code, faculty_id, department, semester) VALUES (?, ?, ?, ?, ?)")
    .run(subjectName, subjectCode || null, facultyId, department, semester);
  res.status(201).json({ success: true, message: "Subject created and assigned.", data: { subjectId: info.lastInsertRowid }
});
};

exports.listSubjects = (req, res) => {
  const rows = db
    .prepare(
      `SELECT s.id, s.subject_name, s.subject_code, s.department, s.semester, f.id AS faculty_id, u.name AS faculty_name
       FROM subjects s JOIN faculty f ON f.id = s.faculty_id JOIN users u ON u.id = f.user_id
       ORDER BY s.department, s.semester, s.subject_name`
    )
    .all();
  res.json({ success: true, data: rows });
};

exports.deleteSubject = (req, res) => {
  db.prepare("DELETE FROM subjects WHERE id = ?").run(req.params.id);
  res.json({ success: true, message: "Subject removed." });
};

/* ---------- Evaluation criteria configuration ---------- */
exports.listCriteria = (req, res) => {
  res.json({ success: true, data: db.prepare("SELECT * FROM criteria ORDER BY sort_order").all() });
};

exports.createCriterion = (req, res) => {
  const { name, weight, sortOrder } = req.body;
  if (!name) return res.status(400).json({ success: false, message: "name is required." });
  const info = db
    .prepare("INSERT INTO criteria (name, weight, sort_order) VALUES (?, ?, ?)")
    .run(name, weight ?? 1.0, sortOrder ?? 0);
  res.status(201).json({ success: true, data: { id: info.lastInsertRowid } });
};

exports.updateCriterion = (req, res) => {
  const { name, weight, isActive, sortOrder } = req.body;
  const current = db.prepare("SELECT * FROM criteria WHERE id = ?").get(req.params.id);
  if (!current) return res.status(404).json({ success: false, message: "Criterion not found." });
  db.prepare("UPDATE criteria SET name=?, weight=?, is_active=?, sort_order=? WHERE id=?").run(
    name ?? current.name,
    weight ?? current.weight,
    isActive === undefined ? current.is_active : (isActive ? 1 : 0),
    sortOrder ?? current.sort_order,
    req.params.id
  );
  res.json({ success: true, message: "Criterion updated." });
};

/* ---------- Feedback period control (start/stop collection) ---------- */
exports.listPeriods = (req, res) => {
  res.json({ success: true, data: db.prepare("SELECT * FROM feedback_periods ORDER BY id DESC").all() });
};

exports.openPeriod = (req, res) => {
  const { academicYear, semester, allowEditing } = req.body;
  if (!academicYear || !semester) {
    return res.status(400).json({ success: false, message: "academicYear and semester are required." });
  }
  db.prepare("UPDATE feedback_periods SET is_open = 0, closed_at = datetime('now') WHERE is_open = 1").run();
  const info = db
    .prepare("INSERT INTO feedback_periods (academic_year, semester, is_open, allow_editing) VALUES (?, ?, 1, ?)")
    .run(academicYear, semester, allowEditing ? 1 : 0);
  res.status(201).json({ success: true, message: "Feedback collection period opened.", data: { periodId: info.lastInsertRow
```

```
id } });
};

exports.closePeriod = (req, res) => {
  db.prepare("UPDATE feedback_periods SET is_open = 0, closed_at = datetime('now') WHERE is_open = 1").run();
  res.json({ success: true, message: "Feedback collection closed." });
};

/* ---------- Report export (CSV) ---------- */
exports.exportAppraisalCsv = (req, res) => {
  const { semester, academicYear } = req.query;
  const period = db.prepare("SELECT * FROM feedback_periods ORDER BY is_open DESC, id DESC LIMIT 1").get();
  const sem = semester || period?.semester;
  const year = academicYear || period?.academic_year;

  const rows = db
    .prepare(
      `SELECT u.name AS faculty_name, f.department, f.designation,
              a.semester, a.academic_year, a.total_responses, a.average_rating, a.appraisal_score, a.grade
       FROM appraisal a
       JOIN faculty f ON f.id = a.faculty_id
       JOIN users u ON u.id = f.user_id
       WHERE a.semester = ? AND a.academic_year = ?
       ORDER BY a.appraisal_score DESC`
    )
    .all(sem, year);

  const header = "Faculty Name,Department,Designation,Semester,Academic Year,Total Responses,Average Rating,Appraisal Score,Grade";
  const csvRows = rows.map((r) =>
    [r.faculty_name, r.department, r.designation, r.semester, r.academic_year, r.total_responses, r.average_rating, r.appraisal_score, r.grade]
      .map((v) => `"${String(v).replace(/"/g, '""')}"`)
      .join(",")
  );
  const csv = [header, ...csvRows].join("\n");

  res.setHeader("Content-Type", "text/csv");
  res.setHeader("Content-Disposition", `attachment; filename="appraisal_report_${year}_sem${sem}.csv"`);
  res.send(csv);
};

```

CHAPTER 5: BACKEND ROUTE DEFINITIONS

API ROUTES

13. Authentication Routes

📁 Source File: `backend/routes/authRoutes.js`

Routes for /api/auth (login, register, me)

```
const express = require("express");
const router = express.Router();
const { login, register, me } = require("../controllers/authController");
const { authenticate } = require("../middleware/auth");

router.post("/login", login);
router.post("/register", register);
router.get("/me", authenticate, me);

module.exports = router;

```

CHAPTER 5: BACKEND ROUTE DEFINITIONS

API ROUTES

14. Feedback Routes

📁 Source File: `backend/routes/feedbackRoutes.js`

Routes for /api/feedback (criteria, targets, status, submit)

```
const express = require("express");
const router = express.Router();
const { authenticate, authorize } = require("../middleware/auth");
const ctrl = require("../controllers/feedbackController");

router.get("/criteria", authenticate, ctrl.getCriteria);
router.get("/targets", authenticate, authorize("student"), ctrl.getTargets);
router.get("/status", authenticate, authorize("student"), ctrl.getStatus);
router.post("/", authenticate, authorize("student"), ctrl.submitFeedback);

module.exports = router;

```

CHAPTER 5: BACKEND ROUTE DEFINITIONS

API ROUTES

15. Faculty Routes

📁 Source File: `backend/routes/facultyRoutes.js`

Routes for /api/faculty (dashboard, performance, comments)

```
const express = require("express");
const router = express.Router();
const { authenticate, authorize } = require("../middleware/auth");
const ctrl = require("../controllers/facultyController");

router.get("/", authenticate, ctrl.listFaculty);
router.get("/:id", authenticate, ctrl.getFacultyById);
router.get("/:id/performance", authenticate, authorize("faculty", "hod"), ctrl.getPerformance);

module.exports = router;

```

CHAPTER 5: BACKEND ROUTE DEFINITIONS

API ROUTES

16. Analytics Routes

📁 Source File: `backend/routes/analyticsRoutes.js`

Routes for /api/analytics (dashboard, comparison, trends)

```
const express = require("express");
const router = express.Router();
const { authenticate, authorize } = require("../middleware/auth");
const ctrl = require("../controllers/analyticsController");

router.use(authenticate, authorize("hod"));
router.get("/dashboard", ctrl.getDashboard);
router.get("/faculty", ctrl.getFacultyComparison);
router.get("/criteria", ctrl.getCriteriaBreakdown);
router.get("/trend", ctrl.getSemesterTrend);
router.get("/recent-activity", ctrl.getRecentActivity);
router.get("/filters", ctrl.getFilterOptions);

module.exports = router;

```

CHAPTER 5: BACKEND ROUTE DEFINITIONS

API ROUTES

17. Appraisal Routes

📁 Source File: `backend/routes/appraisalRoutes.js`

Routes for /api/appraisal (generate, list, export)

```
const express = require("express");
const router = express.Router();
const { authenticate, authorize } = require("../middleware/auth");
const ctrl = require("../controllers/appraisalController");

router.get("/config", authenticate, authorize("hod"), ctrl.getAppraisalConfig);
router.put("/config", authenticate, authorize("hod"), ctrl.updateAppraisalConfig);
router.get("/:facultyId", authenticate, authorize("faculty", "hod"), ctrl.getAppraisal);
router.post("/generate", authenticate, authorize("hod"), ctrl.generateForFacultyOrAll);

module.exports = router;

```

CHAPTER 5: BACKEND ROUTE DEFINITIONS

API ROUTES

18. Admin Management Routes

📁 Source File: `backend/routes/adminRoutes.js`

Routes for /api/admin (students, faculty, subjects, criteria, periods, CSV reports)

```
const express = require("express");
const router = express.Router();
const { authenticate, authorize } = require("../middleware/auth");
const ctrl = require("../controllers/adminController");

router.use(authenticate, authorize("hod"));

router.get("/students", ctrl.listStudents);
router.post("/students", ctrl.createStudent);
router.delete("/students/:id", ctrl.deleteStudent);

router.post("/faculty", ctrl.createFaculty);
router.delete("/faculty/:id", ctrl.deleteFaculty);

router.get("/subjects", ctrl.listSubjects);
router.post("/subjects", ctrl.createSubject);
router.delete("/subjects/:id", ctrl.deleteSubject);

router.get("/criteria", ctrl.listCriteria);
router.post("/criteria", ctrl.createCriterion);
router.put("/criteria/:id", ctrl.updateCriterion);

router.get("/periods", ctrl.listPeriods);
router.post("/periods/open", ctrl.openPeriod);
router.post("/periods/close", ctrl.closePeriod);

router.get("/reports/appraisal.csv", ctrl.exportAppraisalCsv);

module.exports = router;

```

CHAPTER 6: DATA SEEDING & DEPLOYMENT SCRIPTS

SCRIPTS

19. IT Faculty & Subjects Provisioning

📁 Source File: `backend/add_faculty.js`

Auto-creates the 5 Information Technology faculty accounts with assigned semester 5 subjects.

```
require("dotenv").config();
const bcrypt = require("bcryptjs");
const db = require("./config/db");

const facultyData = [
  {
    name: "Shalini Singhal",
    email: "shalinisinghal506@gmail.com",
    department: "Information Technology",
    designation: "Assistant Professor",
    password: "password@123",
    subjectName: "MERN",
    subjectCode: "IT501",
    semester: 5,
  },
  {
    name: "Nikhar Bhatnagar",
    email: "nikharbhatnagar@gmail.com",
    department: "Information Technology",
    designation: "Assistant Professor",
    password: "password@123",
    subjectName: "DAA",
    subjectCode: "IT502",
    semester: 5,
  },
  {
    name: "Jagendra Chaudhary",
    email: "jagendrachaudhary@gmail.com",
    department: "Information Technology",
    designation: "Assistant Professor",
    password: "password@123",
    subjectName: "Cloud Computing",
    subjectCode: "IT503",
    semester: 5,
  },
  {
    name: "Praveen Kumar Yadav",
    email: "praveenkumaryadav@gmail.com",
    department: "Information Technology",
    designation: "Assistant Professor",
    password: "password@123",
    subjectName: "Compiler",
    subjectCode: "IT504",
    semester: 5,
  },
  {
    name: "Manju Chaudhary",
    email: "manjuchaudhary@gmail.com",
    department: "Information Technology",
    designation: "Assistant Professor",
    password: "password@123",
    subjectName: "Operating System",
    subjectCode: "IT505",
    semester: 5,
  },
];

function insertFaculty() {
  console.log("Inserting faculty and subjects...");

  // Update HOD department to Information Technology so it matches
  db.prepare("UPDATE users SET department = 'Information Technology' WHERE role = 'hod'").run();

  // Ensure active feedback period for semester 5 exists
  const existingPeriod = db.prepare("SELECT id FROM feedback_periods WHERE semester = 5 AND is_open = 1").get();
  if (!existingPeriod) {
    db.prepare("UPDATE feedback_periods SET semester = 5 WHERE is_open = 1").run();
  }

```

```
  for (const f of facultyData) {
    const email = f.email.toLowerCase().trim();
    const existing = db.prepare("SELECT id FROM users WHERE email = ?").get(email);

    let userId;
    let facultyId;

    if (existing) {
      userId = existing.id;
      const fac = db.prepare("SELECT id FROM faculty WHERE user_id = ?").get(userId);
      facultyId = fac ? fac.id : null;
      console.log(`User ${f.name} (${email}) already exists.`);
    } else {
      const hash = bcrypt.hashSync(f.password, 10);
      const userRes = db
        .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'faculty', ?)")
        .run(f.name, email, hash, f.department);
      userId = userRes.lastInsertRowid;

      const facRes = db
        .prepare("INSERT INTO faculty (user_id, department, designation) VALUES (?, ?, ?)")
        .run(userId, f.department, f.designation);
      facultyId = facRes.lastInsertRowid;
      console.log(`Added Faculty: ${f.name} (ID: ${facultyId})`);
    }

    if (facultyId && f.subjectName) {
      const existingSubject = db
        .prepare("SELECT id FROM subjects WHERE faculty_id = ? AND subject_name = ?")
        .get(facultyId, f.subjectName);

      if (!existingSubject) {
        db.prepare(
          "INSERT INTO subjects (subject_name, subject_code, faculty_id, department, semester) VALUES (?, ?, ?, ?, ?)"
        ).run(f.subjectName, f.subjectCode, facultyId, f.department, f.semester);
        console.log(`  Assigned Subject: ${f.subjectName} (${f.subjectCode})`);
      }
    }
  }

  console.log("\nAll 5 faculty members and subjects added successfully!");
}

insertFaculty();

```

CHAPTER 6: DATA SEEDING & DEPLOYMENT SCRIPTS

SCRIPTS

20. IT Student Cohort Registration

📁 Source File: `backend/add_students.js`

Registers the IT semester 5 student cohort with secure default credentials.

```
require("dotenv").config();
const bcrypt = require("bcryptjs");
const db = require("./config/db");

const students = [
  {
    name: "Ayush Gautam",
    email: "b240881@skit.ac.in",
    rollNumber: "24ESKIT038",
  },
  {
    name: "Devashish Sharma",
    email: "b240983@skit.ac.in",
    rollNumber: "24ESKIT051",
  },
  {
    name: "Abdud Daiyan",
    email: "b240720@skit.ac.in",
    rollNumber: "24ESKIT002",
  },
];

const DEFAULT_PASSWORD = "password@123";
const DEPARTMENT = "Information Technology";
const SEMESTER = 5;
const SECTION = "A";

function insertStudents() {
  console.log("Inserting students...");
  const hash = bcrypt.hashSync(DEFAULT_PASSWORD, 10);

  for (const s of students) {
    const cleanEmail = s.email.toLowerCase().trim();
    const existing = db.prepare("SELECT id FROM users WHERE email = ?").get(cleanEmail);

    if (existing) {
      console.log(`Student with email ${cleanEmail} already exists, skipping.`);
      continue;
    }

    const tx = db.transaction(() => {
      const userRes = db
        .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'student', ?)")
        .run(s.name, cleanEmail, hash, DEPARTMENT);

      const stuRes = db
        .prepare("INSERT INTO students (user_id, department, semester, section, roll_number) VALUES (?, ?, ?, ?, ?)")
        .run(userRes.lastInsertRowid, DEPARTMENT, SEMESTER, SECTION, s.rollNumber);

      return { userId: userRes.lastInsertRowid, studentId: stuRes.lastInsertRowid };
    });

    const res = tx();
    console.log(`Added student: ${s.name} (${cleanEmail}, Roll: ${s.rollNumber})`);
  }

  console.log("\nAll students inserted successfully!");
}

insertStudents();

```

CHAPTER 6: DATA SEEDING & DEPLOYMENT SCRIPTS

SCRIPTS

21. Database Reset & Clean Utility

📁 Source File: `backend/clean.js`

Wipes demo data while establishing clean tables and the default HOD administrator.

```
require("dotenv").config();
const bcrypt = require("bcryptjs");
const db = require("./config/db");

const DEFAULT_PASSWORD = "Password@123";

function clean() {
  console.log("Cleaning database: removing all demo records...");

  // Wipe all operational and demo data
  db.exec(`
    DELETE FROM feedback_ratings;
    DELETE FROM feedback;
    DELETE FROM appraisal;
    DELETE FROM subjects;
    DELETE FROM faculty;
    DELETE FROM students;
    DELETE FROM users;
    DELETE FROM feedback_periods;
    DELETE FROM criteria;
    DELETE FROM appraisal_config;
  `);

  // Reinitialize default appraisal config
  db.prepare("INSERT INTO appraisal_config (scale_max, pass_threshold, excellent_threshold) VALUES (5, 3.0, 4.5)").run();

  // Reinitialize standard feedback criteria
  const CRITERIA = [
    "Subject Knowledge",
    "Teaching Effectiveness",
    "Communication Skills",
    "Clarity of Explanation",
    "Classroom Management",
    "Punctuality",
    "Use of Teaching Methods",
    "Interaction with Students",
    "Availability and Support",
    "Overall Teaching Performance",
  ];
  const insertCriterion = db.prepare("INSERT INTO criteria (name, weight, sort_order) VALUES (?, 1.0, ?)");
  CRITERIA.forEach((name, i) => insertCriterion.run(name, i));

  // Create clean master HOD account so user can log in and enter everything
  const hash = bcrypt.hashSync(DEFAULT_PASSWORD, 10);
  db.prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'hod', ?)")
    .run("Department HOD", "hod@college.edu", hash, "Computer Science");

  // Create an initial open feedback period
  db.prepare("INSERT INTO feedback_periods (academic_year, semester, is_open, allow_editing) VALUES ('2025-26', 1, 1, 0)").
run();


  console.log("Clean complete! All dummy students, faculty, subjects, and feedbacks removed.");
}

clean();

```

CHAPTER 6: DATA SEEDING & DEPLOYMENT SCRIPTS

SCRIPTS

22. Mock Data Seeder

Source File: `backend/seed.js`

Generates realistic evaluation spread, criteria ratings, and appraisals for demo walkthroughs.

```
/**
 * Seed script — wipes and repopulates the database with realistic demo data:
 * an HOD, 6 faculty across 2 departments, 24 students, subjects, an open
 * feedback period, and randomized-but-plausible feedback so every chart
 * and the appraisal engine has real data to show on first run.
 *
 * Run with: npm run seed
 */
require("dotenv").config();
const bcrypt = require("bcryptjs");
const db = require("./config/db");

const DEFAULT_PASSWORD = "Password@123";

function wipe() {
  db.exec(`
    DELETE FROM feedback_ratings;
    DELETE FROM feedback;
    DELETE FROM appraisal;
    DELETE FROM feedback_periods;
    DELETE FROM subjects;
    DELETE FROM criteria;
    DELETE FROM faculty;
    DELETE FROM students;
    DELETE FROM users;
    DELETE FROM appraisal_config;
  `);
}

function seed() {
  wipe();
  const hash = bcrypt.hashSync(DEFAULT_PASSWORD, 10);

  db.prepare("INSERT INTO appraisal_config (scale_max, pass_threshold, excellent_threshold) VALUES (5, 3.0, 4.5)").run();

  const CRITERIA = [
    "Subject Knowledge",
    "Teaching Effectiveness",
    "Communication Skills",
    "Clarity of Explanation",
    "Classroom Management",
    "Punctuality",
    "Use of Teaching Methods",
    "Interaction with Students",
    "Availability and Support",
    "Overall Teaching Performance",
  ];
  const insertCriterion = db.prepare("INSERT INTO criteria (name, weight, sort_order) VALUES (?, 1.0, ?)");
  const criteriaIds = CRITERIA.map((name, i) => insertCriterion.run(name, i).lastInsertRowid);

  // ---- HOD ----
  const hodUser = db
    .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'hod', ?)")
    .run("Dr. Anjali Mehta", "hod@college.edu", hash, "Computer Science");

  // ---- Departments & Faculty ----
  const departments = ["Computer Science", "Electronics & Communication"];
  const facultyNames = {
    "Computer Science": [
      ["Dr. Ramesh Iyer", "Professor"],
      ["Prof. Sunita Rao", "Associate Professor"],
      ["Dr. Vikram Nair", "Assistant Professor"],
    ],
    "Electronics & Communication": [
      ["Dr. Kavita Sharma", "Professor"],
      ["Prof. Arjun Desai", "Associate Professor"],
      ["Dr. Neha Kulkarni", "Assistant Professor"],
    ],
  };
```

```
    70
    71    const subjectPool = {
    72      "Computer Science": ["Data Structures", "Operating Systems", "Database Systems", "Computer Networks", "Software Enginee
ring", "Artificial Intelligence"],
    73      "Electronics & Communication": ["Digital Electronics", "Signals & Systems", "Microprocessors", "Communication Systems",
"VLSI Design", "Control Systems"],
    74    };
    75
    76    const facultyIds = {}; // name -> id
    77    const subjectsByDept = {};
    78
    79    let emailCounter = 1;
    80    for (const dept of departments) {
    81      subjectsByDept[dept] = [];
    82      facultyNames[dept].forEach(([name, designation], idx) => {
    83        const email = `faculty${emailCounter++}@college.edu`;
    84        const userInfo = db
    85          .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'faculty', ?)")
    86          .run(name, email, hash, dept);
    87        const facInfo = db
    88          .prepare("INSERT INTO faculty (user_id, department, designation) VALUES (?, ?, ?)")
    89          .run(userInfo.lastInsertRowid, dept, designation);
    90        facultyIds[name] = { id: facInfo.lastInsertRowid, email };
    91
    92        // assign 2 subjects per faculty, semester 3 for demo simplicity
    93        const subs = subjectPool[dept].slice(idx * 2, idx * 2 + 2);
    94        subs.forEach((subjectName) => {
    95          const subInfo = db
    96            .prepare("INSERT INTO subjects (subject_name, subject_code, faculty_id, department, semester) VALUES (?, ?, ?, ?,
?)")
    97            .run(subjectName, subjectName.slice(0, 3).toUpperCase() + "301", facInfo.lastInsertRowid, dept, 3);
    98          subjectsByDept[dept].push({ id: subInfo.lastInsertRowid, facultyId: facInfo.lastInsertRowid });
    99        });
   100      });
   101    }
   102
   103    // ---- Feedback period (open) ----
   104    const periodInfo = db
   105      .prepare("INSERT INTO feedback_periods (academic_year, semester, is_open, allow_editing) VALUES ('2025-26', 3, 1, 0)")
   106      .run();
   107    const periodId = periodInfo.lastInsertRowid;
   108
   109    // A closed prior-semester period, so the semester trend chart has 2 points
   110    const prevPeriodInfo = db
   111      .prepare("INSERT INTO feedback_periods (academic_year, semester, is_open, allow_editing, closed_at) VALUES ('2024-25',
2, 0, 0, datetime('now','-6 months'))")
   112      .run();
   113
   114    // ---- Students ----
   115    const studentIdsByDept = {};
   116    let studentEmailCounter = 1;
   117    for (const dept of departments) {
   118      studentIdsByDept[dept] = [];
   119      for (let i = 1; i <= 12; i++) {
   120        const name = `Student ${dept === "Computer Science" ? "CS" : "EC"}-${i}`;
   121        const email = `student${studentEmailCounter++}@college.edu`;
   122        const userInfo = db
   123          .prepare("INSERT INTO users (name, email, password_hash, role, department) VALUES (?, ?, ?, 'student', ?)")
   124          .run(name, email, hash, dept);
   125        const stuInfo = db
   126          .prepare("INSERT INTO students (user_id, department, semester, section, roll_number) VALUES (?, ?, 3, 'A', ?)")
   127          .run(userInfo.lastInsertRowid, dept, `${dept === "Computer Science" ? "CS" : "EC"}25${String(i).padStart(3, "0")}
`);
   128        studentIdsByDept[dept].push({ id: stuInfo.lastInsertRowid, email });
   129      }
   130    }
   131
   132    // ---- Feedback + ratings (current period, with plausible spread) ----
   133    const sampleComments = [
   134      "Explains concepts very clearly with good examples.",
   135      "Classes are engaging and well-paced.",
   136      "Could use more real-world examples.",
   137      "Very approachable during office hours.",
   138      "Sometimes goes too fast through difficult topics.",
   139      "Great use of practical demonstrations.",
   140      "Encourages questions and discussion in class.",
   141      "Notes and slides are well organized.",
   142      "",
   143      "",
   144    ];
```

```
145
146   const insertFeedback = db.prepare(
147     `INSERT INTO feedback (student_id, faculty_id, subject_id, period_id, semester, academic_year, comments) VALUES (?, ?,
?, ?, 3, '2025-26', ?)`
148   );
149   const insertRating = db.prepare("INSERT INTO feedback_ratings (feedback_id, criterion_id, rating) VALUES (?, ?, ?)");
150
151   function randomRatingAround(base) {
152     // base ~3.5-4.7, add noise, clamp 1-5
153     const r = Math.round(base + (Math.random() - 0.5) * 2);
154     return Math.max(1, Math.min(5, r));
155   }
156
157   for (const dept of departments) {
158     const subs = subjectsByDept[dept];
159     const students = studentIdsByDept[dept];
160     subs.forEach((sub, subIdx) => {
161       // Give each faculty a slightly different "quality baseline" so charts look realistic
162       const baseline = 3.4 + ((sub.facultyId * 7) % 10) / 10 + subIdx * 0.05;
163       students.forEach((student, sIdx) => {
164         // ~85% response rate
165         if (Math.random() > 0.85) return;
166         const comment = sampleComments[Math.floor(Math.random() * sampleComments.length)];
167         const fbInfo = insertFeedback.run(student.id, sub.facultyId, sub.id, periodId, comment || null);
168         const feedbackId = fbInfo.lastInsertRowid;
169         criteriaIds.forEach((critId) => {
170           insertRating.run(feedbackId, critId, randomRatingAround(baseline));
171         });
172       });
173     });
174   }
175
176   // ---- Also seed a small set of PRIOR semester appraisal rows directly (for trend chart) ----
177   const insertAppraisal = db.prepare(
178     `INSERT INTO appraisal (faculty_id, semester, academic_year, total_responses, average_rating, appraisal_score, grade) V
ALUES (?, 2, '2024-25', ?, ?, ?, ?)`
179   );
180   Object.values(facultyIds).forEach((f) => {
181     const avg = Number((3.5 + Math.random() * 1.2).toFixed(2));
182     const score = Number(((avg / 5) * 100).toFixed(2));
183     const grade = avg >= 4.5 ? "Excellent" : avg >= 3.0 ? "Satisfactory" : "Needs Improvement";
184     insertAppraisal.run(f.id, 20 + Math.floor(Math.random() * 10), avg, score, grade);
185   });
186
187   // ---- Generate current-period appraisal for every faculty ----
188   const { generateAppraisal } = require("./utils/appraisal");
189   Object.values(facultyIds).forEach((f) => generateAppraisal(f.id, 3, "2025-26"));
190
191   console.log("\n✅ Database seeded successfully!\n");
192   console.log("Demo login credentials (password for ALL accounts): " + DEFAULT_PASSWORD);
193   console.log("-----------------------------------------------------");
194   console.log("HOD / Admin : hod@college.edu");
195   console.log("Faculty     :", Object.values(facultyIds).map((f) => f.email).join(", "));
196   console.log("Students    :", studentIdsByDept["Computer Science"].slice(0, 2).map((s) => s.email).join(", "), "... (24 to
tal)");
197   console.log("-----------------------------------------------------\n");
198 }
199
200 seed();
201
```

CHAPTER 7: FRONTEND APPLICATION CORE & STATE

FRONTEND CORE

23. Vite HTML Entrypoint

📁 Source File: `frontend/index.html`

Single Page Application root HTML template.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=Source+Serif+4:opsz,wght@8..60,400;8..60,500;8..60,600;8..60,700&family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet" />
    <title>Faculty Feedback &amp;amp; Appraisal System</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

CHAPTER 7: FRONTEND APPLICATION CORE & STATE

FRONTEND CORE

24. React Root Mount

📁 Source File: `frontend/src/main.jsx`

Mounts the React root into the DOM with BrowserRouter and AuthProvider.

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./index.css";
import App from "./App.jsx";
import { AuthProvider } from "./context/AuthContext.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>
);

```

CHAPTER 7: FRONTEND APPLICATION CORE & STATE

FRONTEND CORE

25. Application Routes & Role Protection

📁 Source File: `frontend/src/App.jsx`

React Router setup mapping paths to Student, Faculty, and HOD dashboards with route guard protection.

```jsx
import { Routes, Route, Navigate } from "react-router-dom";
import { useAuth } from "./context/AuthContext";
import ProtectedRoute from "./components/ProtectedRoute";
import Login from "./pages/Login";
import StudentDashboard from "./pages/student/StudentDashboard";
import FacultyDashboard from "./pages/faculty/FacultyDashboard";
import HodDashboard from "./pages/hod/HodDashboard";
import HodFacultyPage from "./pages/hod/HodFacultyPage";
import HodStudentsPage from "./pages/hod/HodStudentsPage";
import HodAppraisalsPage from "./pages/hod/HodAppraisalsPage";
import HodReportsPage from "./pages/hod/HodReportsPage";
import HodSettingsPage from "./pages/hod/HodSettingsPage";

export default function App() {
  const { user } = useAuth();

  return (
    <Routes>
      <Route path="/login" element={user ? <Navigate to={`/${user.role}`} replace /> : <Login />} />

      <Route path="/student" element={<ProtectedRoute roles={["student"]}><StudentDashboard /></ProtectedRoute>} />

      <Route path="/faculty" element={<ProtectedRoute roles={["faculty"]}><FacultyDashboard /></ProtectedRoute>} />

      <Route path="/hod" element={<ProtectedRoute roles={["hod"]}><HodDashboard /></ProtectedRoute>} />
      <Route path="/hod/faculty" element={<ProtectedRoute roles={["hod"]}><HodFacultyPage /></ProtectedRoute>} />
      <Route path="/hod/students" element={<ProtectedRoute roles={["hod"]}><HodStudentsPage /></ProtectedRoute>} />
      <Route path="/hod/appraisals" element={<ProtectedRoute roles={["hod"]}><HodAppraisalsPage /></ProtectedRoute>} />
      <Route path="/hod/reports" element={<ProtectedRoute roles={["hod"]}><HodReportsPage /></ProtectedRoute>} />
      <Route path="/hod/settings" element={<ProtectedRoute roles={["hod"]}><HodSettingsPage /></ProtectedRoute>} />

      <Route path="/" element={<Navigate to={user ? `/${user.role}` : "/login"} replace />} />
      <Route path="*" element={<Navigate to={user ? `/${user.role}` : "/login"} replace />} />
    </Routes>
  );
}

```

CHAPTER 7: FRONTEND APPLICATION CORE & STATE

FRONTEND CORE

26. Axios HTTP Client & Auth Interceptor

📁 Source File: `frontend/src/api/client.js`

Centralized API client automatically injecting JWT tokens into request headers.

```
import axios from "axios";

const API_BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:5000/api";

const api = axios.create({ baseURL: API_BASE_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("ffas_token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem("ffas_token");
      localStorage.removeItem("ffas_user");
      if (!window.location.pathname.startsWith("/login")) {
        window.location.href = "/login";
      }
    }
    return Promise.reject(err);
  }
);

export default api;

```

CHAPTER 7: FRONTEND APPLICATION CORE & STATE

FRONTEND CORE

27. Global Authentication Context

📁 Source File: `frontend/src/context/AuthContext.jsx`

React Context managing login, registration, logout, and token synchronization.

```jsx
import { createContext, useContext, useState, useCallback } from "react";
import api from "../api/client";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const raw = localStorage.getItem("ffas_user");
    return raw ? JSON.parse(raw) : null;
  });
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const login = useCallback(async (email, password) => {
    setLoading(true);
    setError("");
    try {
      const { data } = await api.post("/auth/login", { email, password });
      localStorage.setItem("ffas_token", data.data.token);
      localStorage.setItem("ffas_user", JSON.stringify(data.data.user));
      setUser(data.data.user);
      return data.data.user;
    } catch (err) {
      const msg = err.response?.data?.message || "Unable to sign in. Please try again.";
      setError(msg);
      throw new Error(msg);
    } finally {
      setLoading(false);
    }
  }, []);

  const register = useCallback(async (studentData) => {
    setLoading(true);
    setError("");
    try {
      const { data } = await api.post("/auth/register", studentData);
      localStorage.setItem("ffas_token", data.data.token);
      localStorage.setItem("ffas_user", JSON.stringify(data.data.user));
      setUser(data.data.user);
      return data.data.user;
    } catch (err) {
      const msg = err.response?.data?.message || "Unable to register. Please try again.";
      setError(msg);
      throw new Error(msg);
    } finally {
      setLoading(false);
    }
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem("ffas_token");
    localStorage.removeItem("ffas_user");
    setUser(null);
  }, []);

  return (
    <AuthContext.Provider value={{ user, login, register, logout, error, loading, setError }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}

```

CHAPTER 7: FRONTEND APPLICATION CORE & STATE

FRONTEND CORE

28. Tailwind CSS Global Styles

📁 Source File: `frontend/src/index.css`

Custom color palette (ink, ivory, gold, leaf, rust) and reusable utility classes.

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html { -webkit-font-smoothing: antialiased; }
  body {
    @apply bg-paper text-ink-700 font-sans;
  }
  h1, h2, h3, h4 { @apply font-serif text-ink-800; }
  :focus-visible {
    outline: 2px solid theme('colors.gold.500');
    outline-offset: 2px;
  }
}

@layer components {
  .card {
    @apply bg-ivory border border-ink-100 rounded-2xl shadow-card;
  }
  .btn-primary {
    @apply inline-flex items-center justify-center gap-2 rounded-lg bg-ink-700 text-ivory px-4 py-2.5 text-sm font-medium hover:bg-ink-600 transition-colors disabled:opacity-50 disabled:cursor-not-allowed;
  }
  .btn-secondary {
    @apply inline-flex items-center justify-center gap-2 rounded-lg border border-ink-200 bg-ivory text-ink-700 px-4 py-2.5 text-sm font-medium hover:bg-ink-50 transition-colors disabled:opacity-50 disabled:cursor-not-allowed;
  }
  .btn-danger {
    @apply inline-flex items-center justify-center gap-2 rounded-lg bg-rust text-ivory px-4 py-2.5 text-sm font-medium hover:opacity-90 transition-colors;
  }
  .input {
    @apply w-full rounded-lg border border-ink-200 bg-ivory px-3.5 py-2.5 text-sm text-ink-700 placeholder:text-ink-300 focus:border-gold-500 focus:ring-1 focus:ring-gold-500 outline-none transition;
  }
  .label {
    @apply text-sm font-medium text-ink-600 mb-1.5 block;
  }
  .badge {
    @apply inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-medium;
  }
}

```

CHAPTER 8: FRONTEND REUSABLE COMPONENTS

UI COMPONENT

29. App Layout & Navigation Sidebar

📁 Source File: `frontend/src/components/Layout.jsx`

Responsive layout with header, active user badge, sidebar navigation, and logout action.

```jsx
import { NavLink, useNavigate } from "react-router-dom";
import {
  LayoutDashboard,
  Users,
  GraduationCap,
  ClipboardList,
  BarChart3,
  Award,
  FileDown,
  Settings,
  LogOut,
  BookMarked,
} from "lucide-react";
import { useAuth } from "../context/AuthContext";

const NAV = {
  student: [
    { to: "/student", label: "Give Feedback", icon: ClipboardList, end: true },
  ],
  faculty: [
    { to: "/faculty", label: "My Performance", icon: BarChart3, end: true },
  ],
  hod: [
    { to: "/hod", label: "Dashboard", icon: LayoutDashboard, end: true },
    { to: "/hod/faculty", label: "Faculty", icon: GraduationCap },
    { to: "/hod/students", label: "Students", icon: Users },
    { to: "/hod/appraisals", label: "Appraisals", icon: Award },
    { to: "/hod/reports", label: "Reports", icon: FileDown },
    { to: "/hod/settings", label: "Settings", icon: Settings },
  ],
};

export default function Layout({ children, title, subtitle }) {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const items = NAV[user?.role] || [];

  function handleLogout() {
    logout();
    navigate("/login");
  }

  return (
    <div className="min-h-screen flex bg-paper">
      <aside className="hidden md:flex md:w-64 flex-col bg-ink-800 text-ivory shrink-0">
        <div className="flex items-center gap-2.5 px-6 py-6 border-b border-ink-700">
          <div className="h-9 w-9 rounded-lg bg-gold-500 flex items-center justify-center">
            <BookMarked size={18} className="text-ink-800" />
          </div>
          <div>
            <p className="font-serif text-base leading-tight text-ivory">Faculty Feedback</p>
            <p className="text-xs text-ink-300 leading-tight">&amp;amp; Appraisal System</p>
          </div>
        </div>

        <nav className="flex-1 px-3 py-5 space-y-1">
          {items.map(({ to, label, icon: Icon, end }) => (
            <NavLink
              key={to}
              to={to}
              end={end}
              className={({ isActive }) =>
                `flex items-center gap-3 rounded-lg px-3 py-2.5 text-sm font-medium transition-colors ${
                  isActive ? "bg-ink-700 text-ivory" : "text-ink-200 hover:bg-ink-700/60 hover:text-ivory"
                }`
              }
            >
              <Icon size={17} />
              {label}
```

```
              </NavLink>
            ))}
          </nav>

          <div className="px-4 py-4 border-t border-ink-700">
            <div className="mb-3">
              <p className="text-sm font-medium text-ivory truncate">{user?.name}</p>
              <p className="text-xs text-ink-300 truncate">{user?.department}</p>
            </div>
            <button
              onClick={handleLogout}
              className="flex w-full items-center gap-2 rounded-lg px-3 py-2 text-sm text-ink-200 hover:bg-ink-700/60 hover:text-ivory transition-colors"
            >
              <LogOut size={16} /> Sign out
            </button>
          </div>
        </aside>

        <div className="flex-1 min-w-0 flex flex-col">
          <header className="sticky top-0 z-10 bg-paper/95 backdrop-blur border-b border-ink-100 px-5 md:px-8 py-4 flex items-center justify-between">
            <div>
              {title && <h1 className="text-xl md:text-2xl text-ink-800">{title}</h1>}
              {subtitle && <p className="text-sm text-ink-400 mt-0.5">{subtitle}</p>}
            </div>
            <div className="md:hidden flex items-center gap-3">
              <span className="text-sm text-ink-500">{user?.name}</span>
              <button onClick={handleLogout} className="text-ink-500">
                <LogOut size={18} />
              </button>
            </div>
          </header>

          <main className="flex-1 px-5 md:px-8 py-6 max-w-7xl w-full mx-auto">{children}</main>

          <nav className="md:hidden sticky bottom-0 bg-ink-800 border-t border-ink-700 flex justify-around py-2">
            {items.slice(0, 5).map(({ to, label, icon: Icon, end }) => (
              <NavLink
                key={to}
                to={to}
                end={end}
                className={({ isActive }) =>
                  `flex flex-col items-center gap-0.5 px-2 py-1 text-[10px] ${
                    isActive ? "text-gold-500" : "text-ink-300"
                  }`
                }
              >
                <Icon size={18} />
                {label}
              </NavLink>
            ))}
          </nav>
        </div>
      </div>
    );
  }

```

CHAPTER 8: FRONTEND REUSABLE COMPONENTS

UI COMPONENT

30. KPI Card Display

📁 Source File: `frontend/src/components/KPICard.jsx`

Card widget for rendering metric statistics, trends, and change badges.

```
export default function KPICard({ label, value, sublabel, tone = "default", icon: Icon }) {
  const toneClasses = {
    default: "text-ink-800",
    good: "text-leaf",
    warn: "text-gold-600",
    bad: "text-rust",
  };
  return (
    <div className="card p-5">
      <div className="flex items-start justify-between">
        <p className="text-sm font-medium text-ink-400">{label}</p>
        {Icon && (
          <div className="h-8 w-8 rounded-lg bg-ink-50 flex items-center justify-center text-ink-500">
            <Icon size={16} />
          </div>
        )}
      </div>
      <p className={`mt-2 text-3xl font-serif ${toneClasses[tone]}`}>{value}</p>
      {sublabel && <p className="mt-1 text-xs text-ink-400">{sublabel}</p>}
    </div>
  );
}

```

CHAPTER 8: FRONTEND REUSABLE COMPONENTS

UI COMPONENT

31. Star & Numeric Rating Input

📁 Source File: `frontend/src/components/RatingInput.jsx`

Interactive 1 to 5 star rating picker with hover previews and accessibility labels.

```jsx
import { Star } from "lucide-react";

const LABELS = { 1: "Poor", 2: "Below Average", 3: "Average", 4: "Good", 5: "Excellent" };

export default function RatingInput({ value, onChange }) {
  return (
    <div className="flex items-center gap-1.5">
      {[1, 2, 3, 4, 5].map((n) => (
        <button
          key={n}
          type="button"
          onClick={() => onChange(n)}
          aria-label={`${n} - ${LABELS[n]}`}
          className="p-0.5 focus-visible:rounded"
        >
          <Star
            size={26}
            className={n <= value ? "fill-gold-500 text-gold-500" : "text-ink-200"}
            strokeWidth={1.5}
          />
        </button>
      ))}
      <span className="ml-2 text-sm text-ink-400 w-28">{value ? LABELS[value] : "Not rated"}</span>
    </div>
  );
}

```

CHAPTER 8: FRONTEND REUSABLE COMPONENTS

UI COMPONENT

32. Protected Route Barrier

📁 Source File: `frontend/src/components/ProtectedRoute.jsx`

Higher-Order Component guarding routes by role and redirecting unauthorized users.

```jsx
import { Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

export default function ProtectedRoute({ roles, children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.role)) return <Navigate to={`/${user.role}`} replace />;
  return children;
}

```

CHAPTER 8: FRONTEND REUSABLE COMPONENTS

UI COMPONENT

33. Empty State Widget

📁 Source File: `frontend/src/components/EmptyState.jsx`

Placeholder component rendered when queries return zero results.

```jsx
export default function EmptyState({ icon: Icon, title, description, action }) {
  return (
    <div className="card flex flex-col items-center justify-center text-center py-16 px-6">
      {Icon && (
        <div className="h-12 w-12 rounded-full bg-ink-50 flex items-center justify-center text-ink-400 mb-4">
          <Icon size={22} />
        </div>
      )}
      <h3 className="text-lg text-ink-700">{title}</h3>
      {description && <p className="mt-1.5 text-sm text-ink-400 max-w-sm">{description}</p>}
      {action && <div className="mt-5">{action}</div>}
    </div>
  );
}

```

CHAPTER 8: FRONTEND REUSABLE COMPONENTS

UI COMPONENT

34. Toast Notification Widget

📁 Source File: `frontend/src/components/Toast.jsx`

Non-intrusive floating alert messages for action confirmations.

```jsx
import { useEffect } from "react";
import { CheckCircle2, XCircle, X } from "lucide-react";

export default function Toast({ toast, onClose }) {
  useEffect(() => {
    if (!toast) return;
    const t = setTimeout(onClose, 4000);
    return () => clearTimeout(t);
  }, [toast, onClose]);

  if (!toast) return null;
  const isError = toast.type === "error";

  return (
    <div className="fixed bottom-6 right-6 z-50 animate-[fadeIn_0.2s_ease-out]">
      <div
        className={`flex items-start gap-3 rounded-xl border px-4 py-3.5 shadow-lg max-w-sm ${
          isError ? "bg-rust/5 border-rust/20" : "bg-leaf/5 border-leaf/20"
        }`}
      >
        {isError ? (
          <XCircle size={20} className="text-rust shrink-0 mt-0.5" />
        ) : (
          <CheckCircle2 size={20} className="text-leaf shrink-0 mt-0.5" />
        )}
        <p className="text-sm text-ink-700">{toast.message}</p>
        <button onClick={onClose} className="ml-auto text-ink-300 hover:text-ink-500">
          <X size={16} />
        </button>
      </div>
    </div>
  );
}

```

CHAPTER 9: STUDENT PORTAL PAGES

STUDENT PAGE

35. Student Dashboard

📁 Source File: `frontend/src/pages/student/StudentDashboard.jsx`

Displays assigned semester subjects, pending feedback cards, and overall submission progress.

```
import { useEffect, useState } from "react";
import { CheckCircle2, Circle, ClipboardCheck, Loader2 } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";
import EmptyState from "../../components/EmptyState";
import FeedbackForm from "./FeedbackForm";

export default function StudentDashboard() {
  const [loading, setLoading] = useState(true);
  const [data, setData] = useState(null);
  const [status, setStatus] = useState(null);
  const [activeTarget, setActiveTarget] = useState(null);

  async function load() {
    setLoading(true);
    const [targetsRes, statusRes] = await Promise.all([
      api.get("/feedback/targets"),
      api.get("/feedback/status"),
    ]);
    setData(targetsRes.data.data);
    setStatus(statusRes.data.data);
    setLoading(false);
  }

  useEffect(() => {
    load();
  }, []);

  if (loading) {
    return (
      <Layout title="Give Feedback">
        <div className="flex items-center justify-center py-24 text-ink-400">
          <Loader2 className="animate-spin mr-2" size={18} /> Loading your subjects…
        </div>
      </Layout>
    );
  }

  if (activeTarget) {
    return (
      <FeedbackForm
        target={activeTarget}
        allowEditing={data.allowEditing}
        onDone={() => {
          setActiveTarget(null);
          load();
        }}
        onCancel={() => setActiveTarget(null)}
      />
    );
  }

  return (
    <Layout
      title="Give Feedback"
      subtitle={data.periodOpen ? `Feedback collection is open for ${data.period.academic_year}, Semester ${data.period.sem
ester}` : undefined}
    >
      {!data.periodOpen ? (
        <EmptyState
          icon={ClipboardCheck}
          title="Feedback collection is currently closed"
          description="Your department has not opened a feedback window for this semester yet. Please check back later."
        />
      ) : (
        <>
          <div className="card p-5 mb-6">
            <div className="flex items-center justify-between mb-2">
              <p className="text-sm font-medium text-ink-600">Your completion progress</p>
```

```
              <p className="text-sm text-ink-400">{status.completed} of {status.totalSubjects} completed</p>
            </div>
            <div className="h-2.5 rounded-full bg-ink-100 overflow-hidden">
              <div
                className="h-full rounded-full bg-gold-500 transition-all duration-500"
                style={{ width: `${status.percentage}%` }}
              />
            </div>
          </div>

          <div className="space-y-3">
            {data.targets.map((t) => (
              <div key={`${t.faculty_id}-${t.subject_id}`} className="card p-4 flex items-center justify-between gap-4">
                <div className="flex items-center gap-3 min-w-0">
                  {t.submitted ? (
                    <CheckCircle2 size={20} className="text-leaf shrink-0" />
                  ) : (
                    <Circle size={20} className="text-ink-200 shrink-0" />
                  )}
                  <div className="min-w-0">
                    <p className="text-sm font-medium text-ink-700 truncate">{t.subject_name}</p>
                    <p className="text-xs text-ink-400 truncate">{t.faculty_name} · {t.designation}</p>
                  </div>
                </div>
                <button
                  onClick={() => setActiveTarget(t)}
                  disabled={t.submitted && !data.allowEditing}
                  className={t.submitted ? "btn-secondary shrink-0" : "btn-primary shrink-0"}
                >
                  {t.submitted ? (data.allowEditing ? "Edit" : "Submitted") : "Give Feedback"}
                </button>
              </div>
            ))}
          </div>
        </>
      )}
    </Layout>
  );
}
```

CHAPTER 9: STUDENT PORTAL PAGES

STUDENT PAGE

36. Interactive Feedback Submission Form

📁 Source File: `frontend/src/pages/student/FeedbackForm.jsx`

Dynamic questionnaire form rendering all active evaluation criteria, star pickers, comments, and submission handler.

```jsx
import { useEffect, useState } from "react";
import { ArrowLeft, CheckCircle2, Loader2 } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";
import RatingInput from "../../components/RatingInput";

export default function FeedbackForm({ target, onDone, onCancel }) {
  const [criteria, setCriteria] = useState([]);
  const [ratings, setRatings] = useState({});
  const [comments, setComments] = useState("");
  const [loading, setLoading] = useState(true);
  const [submitting, setSubmitting] = useState(false);
  const [submitted, setSubmitted] = useState(false);
  const [error, setError] = useState("");

  useEffect(() => {
    api.get("/feedback/criteria").then((res) => {
      setCriteria(res.data.data);
      setLoading(false);
    });
  }, []);

  const allRated = criteria.length > 0 && criteria.every((c) => ratings[c.id]);

  async function handleSubmit() {
    setError("");
    setSubmitting(true);
    try {
      await api.post("/feedback", {
        facultyId: target.faculty_id,
        subjectId: target.subject_id,
        ratings: criteria.map((c) => ({ criterionId: c.id, rating: ratings[c.id] })),
        comments: comments.trim() || undefined,
      });
      setSubmitted(true);
    } catch (err) {
      setError(err.response?.data?.message || "Something went wrong while submitting your feedback.");
    } finally {
      setSubmitting(false);
    }
  }

  if (submitted) {
    return (
      <Layout title="Give Feedback">
        <div className="card flex flex-col items-center justify-center text-center py-20 px-6">
          <div className="h-14 w-14 rounded-full bg-leaf/10 flex items-center justify-center text-leaf mb-5">
            <CheckCircle2 size={28} />
          </div>
          <h2 className="text-xl text-ink-800">Feedback submitted successfully</h2>
          <p className="mt-2 text-sm text-ink-400 max-w-sm">
            Thank you for contributing to academic improvement.
          </p>
          <button onClick={onDone} className="btn-primary mt-6">Back to my subjects</button>
        </div>
      </Layout>
    );
  }

  return (
    <Layout title={target.subject_name} subtitle={`Feedback for ${target.faculty_name} · ${target.designation}`}>
      <button onClick={onCancel} className="flex items-center gap-1.5 text-sm text-ink-400 hover:text-ink-600 mb-5">
        <ArrowLeft size={15} /> Back to my subjects
      </button>

      {loading ? (
        <div className="flex items-center justify-center py-20 text-ink-400">
          <Loader2 className="animate-spin mr-2" size={18} /> Loading feedback form…
        </div>
```

```
      ) : (
        <div className="card p-6">
          <p className="text-sm text-ink-500 mb-6">
            Rate each criterion from 1 (Poor) to 5 (Excellent). Your responses are aggregated and never shown to faculty individually.
          </p>

          <div className="space-y-5 divide-y divide-ink-100">
            {criteria.map((c) => (
              <div key={c.id} className="pt-5 first:pt-0 flex flex-col sm:flex-row sm:items-center sm:justify-between gap-2">
                <p className="text-sm font-medium text-ink-700">{c.name}</p>
                <RatingInput value={ratings[c.id] || 0} onChange={(v) => setRatings((r) => ({ ...r, [c.id]: v }))} />
              </div>
            ))}
          </div>

          <div className="mt-6">
            <label className="label" htmlFor="comments">Additional comments (optional)</label>
            <textarea
              id="comments"
              className="input min-h-[100px] resize-y"
              placeholder="Share any specific feedback that could help this faculty member improve…"
              value={comments}
              onChange={(e) => setComments(e.target.value)}
              maxLength={1000}
            />
          </div>

          {error && (
            <p className="mt-4 text-sm text-rust bg-rust/5 border border-rust/20 rounded-lg px-3 py-2">{error}</p>
          )}

          <div className="mt-6 flex items-center gap-3">
            <button onClick={handleSubmit} disabled={!allRated || submitting} className="btn-primary">
              {submitting && <Loader2 size={16} className="animate-spin" />}
              Submit Feedback
            </button>
            {!allRated && <p className="text-xs text-ink-400">Please rate every criterion to continue.</p>}
          </div>
        </div>
      )}
    </Layout>
  );
}
```

CHAPTER 10: FACULTY PORTAL PAGES

FACULTY PAGE

37. Faculty Analytics & Appraisal Dashboard

📁 Source File: `frontend/src/pages/faculty/FacultyDashboard.jsx`

Visualizes the faculty member's appraisal score, performance grade, criterion-by-criterion bar breakdown, and student comments.

```jsx
import { useEffect, useState } from "react";
import {
  RadarChart, PolarGrid, PolarAngleAxis, PolarRadiusAxis, Radar,
  LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer,
  BarChart, Bar, Cell,
} from "recharts";
import { Award, MessageSquareText, Star, TrendingUp, Loader2, Info } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";
import KPICard from "../../components/KPICard";
import EmptyState from "../../components/EmptyState";
import { useAuth } from "../../context/AuthContext";

const GRADE_TONE = {
  Excellent: "good",
  Satisfactory: "default",
  "Needs Improvement": "bad",
};

export default function FacultyDashboard({ facultyIdOverride }) {
  const { user } = useAuth();
  const facultyId = facultyIdOverride || user.profileId;
  const [loading, setLoading] = useState(true);
  const [data, setData] = useState(null);

  useEffect(() => {
    setLoading(true);
    api.get(`/faculty/${facultyId}/performance`).then((res) => {
      setData(res.data.data);
      setLoading(false);
    });
  }, [facultyId]);

  if (loading) {
    return (
      <Layout title="My Performance">
        <div className="flex items-center justify-center py-24 text-ink-400">
          <Loader2 className="animate-spin mr-2" size={18} /> Loading performance data…
        </div>
      </Layout>
    );
  }

  const { faculty, appraisal, trend, ratingDistribution, comments } = data;

  if (!appraisal || appraisal.totalResponses === 0) {
    return (
      <Layout title="My Performance" subtitle={`${faculty.name} · ${faculty.department}`}>
        <EmptyState
          icon={Info}
          title="No feedback received yet"
          description="Once students submit feedback for this semester, your performance summary, appraisal score and charts will appear here."
        />
      </Layout>
    );
  }

  const radarData = appraisal.criterionAverages.map((c) => ({ criterion: c.name.replace(/ /g, "\n"), rating: c.averageRating }));
  const distData = [1, 2, 3, 4, 5].map((n) => ({
    rating: `${n}`,
    count: ratingDistribution.find((r) => r.rating === n)?.c || 0,
  }));
  const trendData = trend.map((t) => ({ label: `${t.academic_year} S${t.semester}`, rating: t.average_rating }));

  return (
    <Layout title="My Performance" subtitle={`${faculty.name} · ${faculty.department} · ${faculty.designation}`}>
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4 mb-6">
```

```
          <KPICard label="Overall Rating" value={`${appraisal.weightedAverageRating} / ${appraisal.scaleMax}`} icon={Star} />
          <KPICard
            label="Appraisal Score"
            value={`${appraisal.appraisalScore}%`}
            sublabel={appraisal.grade}
            tone={GRADE_TONE[appraisal.grade]}
            icon={Award}
          />
          <KPICard label="Responses" value={appraisal.totalResponses} icon={MessageSquareText} />
          <KPICard
            label="Semester"
            value={`Sem ${appraisal.semester}`}
            sublabel={appraisal.academicYear}
            icon={TrendingUp}
          />
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 mb-6">
          <div className="card p-5">
            <h3 className="text-base text-ink-700 mb-1">Criterion-wise Performance</h3>
            <p className="text-xs text-ink-400 mb-4">Average rating (1–5) across each evaluation criterion</p>
            <ResponsiveContainer width="100%" height={300}>
              <RadarChart data={radarData} outerRadius="75%">
                <PolarGrid stroke="#D9E0EC" />
                <PolarAngleAxis dataKey="criterion" tick={{ fontSize: 10, fill: "#4C6291" }} />
                <PolarRadiusAxis angle={90} domain={[0, 5]} tick={{ fontSize: 9, fill: "#7E92B8" }} />
                <Radar dataKey="rating" stroke="#C08A28" fill="#C08A28" fillOpacity={0.35} strokeWidth={2} />
                <Tooltip />
              </RadarChart>
            </ResponsiveContainer>
          </div>

          <div className="card p-5">
            <h3 className="text-base text-ink-700 mb-1">Semester Performance Trend</h3>
            <p className="text-xs text-ink-400 mb-4">Average rating across academic terms</p>
            <ResponsiveContainer width="100%" height={300}>
              <LineChart data={trendData} margin={{ left: -10 }}>
                <CartesianGrid stroke="#EEF1F7" vertical={false} />
                <XAxis dataKey="label" tick={{ fontSize: 11, fill: "#4C6291" }} />
                <YAxis domain={[0, 5]} tick={{ fontSize: 11, fill: "#4C6291" }} />
                <Tooltip />
                <Line type="monotone" dataKey="rating" stroke="#2F4373" strokeWidth={2.5} dot={{ r: 4, fill: "#2F4373" }} />
              </LineChart>
            </ResponsiveContainer>
          </div>
        </div>

        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 mb-6">
          <div className="card p-5">
            <h3 className="text-base text-ink-700 mb-1">Rating Distribution</h3>
            <p className="text-xs text-ink-400 mb-4">How many individual ratings fell at each score, this semester</p>
            <ResponsiveContainer width="100%" height={260}>
              <BarChart data={distData} margin={{ left: -10 }}>
                <CartesianGrid stroke="#EEF1F7" vertical={false} />
                <XAxis dataKey="rating" tick={{ fontSize: 11, fill: "#4C6291" }} />
                <YAxis tick={{ fontSize: 11, fill: "#4C6291" }} allowDecimals={false} />
                <Tooltip />
                <Bar dataKey="count" radius={[6, 6, 0, 0]}>
                  {distData.map((d, i) => (
                    <Cell key={i} fill={["#B4483B", "#C08A28", "#DDB165", "#3F7D5C", "#2F4373"][i]} />
                  ))}
                </Bar>
              </BarChart>
            </ResponsiveContainer>
          </div>

          <div className="card p-5">
            <h3 className="text-base text-ink-700 mb-1">How your score is calculated</h3>
            <p className="text-xs text-ink-400 mb-4">Transparent appraisal methodology</p>
            <ol className="space-y-3 text-sm text-ink-600">
              <li className="flex gap-2.5">
                <span className="shrink-0 h-5 w-5 rounded-full bg-ink-50 text-ink-500 text-xs flex items-center justify-cente
r font-medium">1</span>
                Each criterion's average is the mean of all student ratings for that criterion.
              </li>
              <li className="flex gap-2.5">
                <span className="shrink-0 h-5 w-5 rounded-full bg-ink-50 text-ink-500 text-xs flex items-center justify-cente
r font-medium">2</span>
                Criterion averages are combined into one weighted average rating (currently equal weights).
              </li>
```

```
146              <li className="flex gap-2.5">
147                <span className="shrink-0 h-5 w-5 rounded-full bg-ink-50 text-ink-500 text-xs flex items-center justify-cente
r font-medium">3</span>
148                That average is scaled to a 0–100 appraisal score: (average ÷ {appraisal.scaleMax}) × 100.
149              </li>
150            </ol>
151            <div className="mt-5 pt-4 border-t border-ink-100 flex items-center justify-between text-sm">
152              <span className="text-ink-500">Satisfactory threshold</span>
153              <span className="text-ink-700 font-medium">{appraisal.methodology.thresholds.satisfactory} / 5</span>
154            </div>
155            <div className="flex items-center justify-between text-sm mt-1.5">
156              <span className="text-ink-500">Excellent threshold</span>
157              <span className="text-ink-700 font-medium">{appraisal.methodology.thresholds.excellent} / 5</span>
158            </div>
159          </div>
160        </div>
161
162        <div className="card p-5">
163          <h3 className="text-base text-ink-700 mb-1">Aggregated Student Comments</h3>
164          <p className="text-xs text-ink-400 mb-4">Shown without any identifying information</p>
165          {comments.length === 0 ? (
166            <p className="text-sm text-ink-400 py-6 text-center">No written comments were submitted this semester.</p>
167          ) : (
168            <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
169              {comments.map((c, i) => (
170                <div key={i} className="bg-ink-50 rounded-lg px-4 py-3 text-sm text-ink-600">
171                  “{c}”
172                </div>
173              ))}
174            </div>
175          )}
176        </div>
177      </Layout>
178    );
179  }
180
```

CHAPTER 11: HOD & ADMINISTRATION PAGES

HOD PAGE

38. HOD Executive Analytics Dashboard

📁 Source File: `frontend/src/pages/hod/HodDashboard.jsx`

Department-wide overview featuring feedback completion gauges, faculty ranking table, and performance charts.

```jsx
import { useEffect, useState, useCallback } from "react";
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Cell,
  RadarChart, PolarGrid, PolarAngleAxis, PolarRadiusAxis, Radar,
  LineChart, Line,
} from "recharts";
import { Users, GraduationCap, ClipboardCheck, Star, Loader2, Clock } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";
import KPICard from "../../components/KPICard";

function barColor(rating) {
  if (rating >= 4.5) return "#3F7D5C";
  if (rating >= 3) return "#C08A28";
  return "#B4483B";
}

export default function HodDashboard() {
  const [filters, setFilters] = useState(null);
  const [department, setDepartment] = useState("");
  const [loading, setLoading] = useState(true);
  const [kpi, setKpi] = useState(null);
  const [facultyComparison, setFacultyComparison] = useState([]);
  const [criteria, setCriteria] = useState([]);
  const [trend, setTrend] = useState([]);
  const [activity, setActivity] = useState([]);

  const loadAll = useCallback(async (dept) => {
    setLoading(true);
    const params = dept ? { department: dept } : {};
    const [dashRes, facRes, critRes, trendRes, actRes] = await Promise.all([
      api.get("/analytics/dashboard", { params }),
      api.get("/analytics/faculty", { params }),
      api.get("/analytics/criteria", { params }),
      api.get("/analytics/trend", { params }),
      api.get("/analytics/recent-activity"),
    ]);
    setKpi(dashRes.data.data);
    setFacultyComparison(facRes.data.data);
    setCriteria(critRes.data.data);
    setTrend(trendRes.data.data);
    setActivity(actRes.data.data);
    setLoading(false);
  }, []);

  useEffect(() => {
    api.get("/analytics/filters").then((res) => setFilters(res.data.data));
    loadAll("");
  }, [loadAll]);

  function handleDeptChange(e) {
    const dept = e.target.value;
    setDepartment(dept);
    loadAll(dept);
  }

  return (
    <Layout title="Faculty Performance Overview" subtitle="Department-wide feedback and appraisal analytics">
      <div className="flex flex-wrap items-center gap-3 mb-6">
        <select value={department} onChange={handleDeptChange} className="input w-auto min-w-[220px]">
          <option value="">All Departments</option>
          {filters?.departments.map((d) => (
            <option key={d} value={d}>{d}</option>
          ))}
        </select>
        {kpi && <span className="text-xs text-ink-400">Showing {kpi.academicYear} · Semester {kpi.semester}</span>}
      </div>

      {loading || !kpi ? (
```

```
          <div className="flex items-center justify-center py-24 text-ink-400">
            <Loader2 className="animate-spin mr-2" size={18} /> Loading analytics…
          </div>
        ) : (
          <>
            <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4 mb-6">
              <KPICard label="Total Faculty" value={kpi.totalFaculty} icon={GraduationCap} />
              <KPICard label="Total Students" value={kpi.totalStudents} icon={Users} />
              <KPICard label="Feedback Completion" value={`${kpi.completionPercentage}%`} sublabel={`${kpi.totalFeedback} responses received`} icon={ClipboardCheck} />
              <KPICard label="Department Average" value={`${kpi.departmentAverageRating} / 5`} icon={Star} tone={kpi.departmentAverageRating >= 4 ? "good" : kpi.departmentAverageRating >= 3 ? "default" : "bad"} />
            </div>

            <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 mb-6">
              <div className="card p-5">
                <h3 className="text-base text-ink-700 mb-1">Faculty Rating Comparison</h3>
                <p className="text-xs text-ink-400 mb-4">Average rating per faculty member, this semester</p>
                <ResponsiveContainer width="100%" height={Math.max(260, facultyComparison.length * 38)}>
                  <BarChart data={facultyComparison} layout="vertical" margin={{ left: 24 }}>
                    <CartesianGrid stroke="#EEF1F7" horizontal={false} />
                    <XAxis type="number" domain={[0, 5]} tick={{ fontSize: 11, fill: "#4C6291" }} />
                    <YAxis type="category" dataKey="name" width={140} tick={{ fontSize: 11, fill: "#2F4373" }} />
                    <Tooltip />
                    <Bar dataKey="averageRating" radius={[0, 6, 6, 0]} barSize={18}>
                      {facultyComparison.map((f, i) => (
                        <Cell key={i} fill={barColor(f.averageRating)} />
                      ))}
                    </Bar>
                  </BarChart>
                </ResponsiveContainer>
              </div>

              <div className="card p-5">
                <h3 className="text-base text-ink-700 mb-1">Criterion-wise Performance</h3>
                <p className="text-xs text-ink-400 mb-4">Department average across evaluation criteria</p>
                <ResponsiveContainer width="100%" height={300}>
                  <RadarChart data={criteria.map((c) => ({ criterion: c.criterion.replace(/ /g, "\n"), rating: c.averageRating }))} outerRadius="75%">
                    <PolarGrid stroke="#D9E0EC" />
                    <PolarAngleAxis dataKey="criterion" tick={{ fontSize: 10, fill: "#4C6291" }} />
                    <PolarRadiusAxis angle={90} domain={[0, 5]} tick={{ fontSize: 9, fill: "#7E92B8" }} />
                    <Radar dataKey="rating" stroke="#2F4373" fill="#2F4373" fillOpacity={0.3} strokeWidth={2} />
                    <Tooltip />
                  </RadarChart>
                </ResponsiveContainer>
              </div>
            </div>

            <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
              <div className="card p-5 lg:col-span-2">
                <h3 className="text-base text-ink-700 mb-1">Semester Trend</h3>
                <p className="text-xs text-ink-400 mb-4">Average departmental rating over academic terms</p>
                <ResponsiveContainer width="100%" height={240}>
                  <LineChart data={trend.map((t) => ({ label: t.label, rating: t.averageRating }))} margin={{ left: -10 }}>
                    <CartesianGrid stroke="#EEF1F7" vertical={false} />
                    <XAxis dataKey="label" tick={{ fontSize: 11, fill: "#4C6291" }} />
                    <YAxis domain={[0, 5]} tick={{ fontSize: 11, fill: "#4C6291" }} />
                    <Tooltip />
                    <Line type="monotone" dataKey="rating" stroke="#C08A28" strokeWidth={2.5} dot={{ r: 4, fill: "#C08A28" }} />
                  </LineChart>
                </ResponsiveContainer>
              </div>

              <div className="card p-5">
                <h3 className="text-base text-ink-700 mb-1">Recent Feedback Activity</h3>
                <p className="text-xs text-ink-400 mb-4">Latest submissions across the department</p>
                <div className="space-y-3 max-h-[240px] overflow-y-auto pr-1">
                  {activity.length === 0 && <p className="text-sm text-ink-400">No activity yet.</p>}
                  {activity.map((a, i) => (
                    <div key={i} className="flex items-start gap-2.5 text-sm">
                      <Clock size={14} className="text-ink-300 mt-0.5 shrink-0" />
                      <div className="min-w-0">
                        <p className="text-ink-600 truncate">{a.faculty_name} · {a.subject_name}</p>
                        <p className="text-xs text-ink-400">{new Date(a.submitted_at).toLocaleString()}</p>
                      </div>
                    </div>
                  ))}
                </div>
```

```
            </div>
          </div>
        </>
      )}
    </Layout>
  );
}

```

CHAPTER 11: HOD & ADMINISTRATION PAGES

HOD PAGE

39. Faculty Roster & Assignment Page

📁 Source File: `frontend/src/pages/hod/HodFacultyPage.jsx`

HOD view to add new faculty members, inspect current faculty list, and assign subjects.

```jsx
import { useEffect, useState } from "react";
import { ArrowLeft, ChevronRight, Loader2, Plus, Trash2, X } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";
import FacultyDashboard from "../faculty/FacultyDashboard";

export default function HodFacultyPage() {
  const [faculty, setFaculty] = useState([]);
  const [loading, setLoading] = useState(true);
  const [selected, setSelected] = useState(null);
  const [showAdd, setShowAdd] = useState(false);
  const [form, setForm] = useState({ name: "", email: "", password: "Password@123", department: "", designation: "Assistant Professor" });
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState("");

  function load() {
    setLoading(true);
    api.get("/faculty").then((res) => {
      setFaculty(res.data.data);
      setLoading(false);
    });
  }

  useEffect(load, []);

  async function handleAdd(e) {
    e.preventDefault();
    setSaving(true);
    setError("");
    try {
      await api.post("/admin/faculty", form);
      setShowAdd(false);
      setForm({ name: "", email: "", password: "Password@123", department: "", designation: "Assistant Professor" });
      load();
    } catch (err) {
      setError(err.response?.data?.message || "Could not add faculty member.");
    } finally {
      setSaving(false);
    }
  }

  async function handleDelete(id, name) {
    if (!window.confirm(`Remove ${name} from the system? This cannot be undone.`)) return;
    await api.delete(`/admin/faculty/${id}`);
    load();
  }

  if (selected) {
    return (
      <div>
        <div className="px-5 md:px-8 pt-6">
          <button
            onClick={() => setSelected(null)}
            className="flex items-center gap-1.5 text-sm text-ink-400 hover:text-ink-600"
          >
            <ArrowLeft size={15} /> Back to faculty list
          </button>
        </div>
        <FacultyDashboard facultyIdOverride={selected} />
      </div>
    );
  }

  return (
    <Layout title="Faculty" subtitle="Manage faculty and drill into individual performance">
      <div className="flex justify-end mb-5">
        <button onClick={() => setShowAdd(true)} className="btn-primary">
          <Plus size={16} /> Add Faculty
```

```
          </button>
        </div>

        {loading ? (
          <div className="flex items-center justify-center py-24 text-ink-400">
            <Loader2 className="animate-spin mr-2" size={18} /> Loading faculty…
          </div>
        ) : (
          <div className="card divide-y divide-ink-100">
            {faculty.map((f) => (
              <div key={f.id} className="p-4 flex items-center justify-between gap-3 hover:bg-ink-50/50 transition-colors">
                <button onClick={() => setSelected(f.id)} className="flex-1 flex items-center justify-between text-left min-w
-0">
                  <div className="min-w-0">
                    <p className="text-sm font-medium text-ink-700 truncate">{f.name}</p>
                    <p className="text-xs text-ink-400 truncate">{f.department} · {f.designation}</p>
                  </div>
                  <ChevronRight size={18} className="text-ink-300 shrink-0" />
                </button>
                <button onClick={() => handleDelete(f.id, f.name)} className="text-ink-300 hover:text-rust p-1.5 shrink-0">
                  <Trash2 size={15} />
                </button>
              </div>
            ))}
          </div>
        )}

        {showAdd && (
          <div className="fixed inset-0 bg-ink-900/40 flex items-center justify-center px-4 z-50">
            <div className="card w-full max-w-md p-6 bg-ivory">
              <div className="flex items-center justify-between mb-4">
                <h3 className="text-lg text-ink-800">Add Faculty Member</h3>
                <button onClick={() => setShowAdd(false)} className="text-ink-300 hover:text-ink-500"><X size={18} /></button
>
              </div>
              <form onSubmit={handleAdd} className="space-y-3">
                <div>
                  <label className="label">Full name</label>
                  <input required className="input" value={form.name} onChange={(e) => setForm({ ...form, name: e.target.valu
e })} />
                </div>
                <div>
                  <label className="label">Email</label>
                  <input required type="email" className="input" value={form.email} onChange={(e) => setForm({ ...form, emai
l: e.target.value })} />
                </div>
                <div>
                  <label className="label">Department</label>
                  <input required className="input" value={form.department} onChange={(e) => setForm({ ...form, department:
e.target.value })} />
                </div>
                <div>
                  <label className="label">Designation</label>
                  <input className="input" value={form.designation} onChange={(e) => setForm({ ...form, designation: e.targe
t.value })} />
                </div>
                {error && <p className="text-sm text-rust">{error}</p>}
                <div className="flex gap-3 pt-2">
                  <button type="submit" disabled={saving} className="btn-primary flex-1">
                    {saving && <Loader2 size={16} className="animate-spin" />} Add Faculty
                  </button>
                  <button type="button" onClick={() => setShowAdd(false)} className="btn-secondary flex-1">Cancel</button>
                </div>
              </form>
            </div>
          </div>
        )}
      </Layout>
    );
  }
```

CHAPTER 11: HOD & ADMINISTRATION PAGES

HOD PAGE

40. Student Roster Management Page

📁 Source File: `frontend/src/pages/hod/HodStudentsPage.jsx`

Enrolled student management interface for viewing student lists and manual registrations.

```
import { useEffect, useState } from "react";
import { Loader2, Plus, Trash2, X } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";

export default function HodStudentsPage() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showAdd, setShowAdd] = useState(false);
  const [form, setForm] = useState({ name: "", email: "", password: "Password@123", department: "", semester: 3, section: "A", rollNumber: "" });
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState("");

  function load() {
    setLoading(true);
    api.get("/admin/students").then((res) => {
      setStudents(res.data.data);
      setLoading(false);
    });
  }
  useEffect(load, []);

  async function handleAdd(e) {
    e.preventDefault();
    setSaving(true);
    setError("");
    try {
      await api.post("/admin/students", { ...form, semester: Number(form.semester) });
      setShowAdd(false);
      setForm({ name: "", email: "", password: "Password@123", department: "", semester: 3, section: "A", rollNumber: "" });
      load();
    } catch (err) {
      setError(err.response?.data?.message || "Could not add student.");
    } finally {
      setSaving(false);
    }
  }

  async function handleDelete(id, name) {
    if (!window.confirm(`Remove ${name}? This cannot be undone.`)) return;
    await api.delete(`/admin/students/${id}`);
    load();
  }

  return (
    <Layout title="Students" subtitle="Manage enrolled students">
      <div className="flex justify-end mb-5">
        <button onClick={() => setShowAdd(true)} className="btn-primary"><Plus size={16} /> Add Student</button>
      </div>

      {loading ? (
        <div className="flex items-center justify-center py-24 text-ink-400">
          <Loader2 className="animate-spin mr-2" size={18} /> Loading students…
        </div>
      ) : (
        <div className="card overflow-x-auto">
          <table className="w-full text-sm">
            <thead>
              <tr className="border-b border-ink-100 text-left text-ink-400">
                <th className="px-4 py-3 font-medium">Name</th>
                <th className="px-4 py-3 font-medium">Roll No.</th>
                <th className="px-4 py-3 font-medium">Department</th>
                <th className="px-4 py-3 font-medium">Semester</th>
                <th className="px-4 py-3 font-medium">Section</th>
                <th className="px-4 py-3"></th>
              </tr>
            </thead>
```

```
            <tbody className="divide-y divide-ink-100">
              {students.map((s) => (
                <tr key={s.id} className="hover:bg-ink-50/50">
                  <td className="px-4 py-3 text-ink-700">{s.name}</td>
                  <td className="px-4 py-3 text-ink-500">{s.roll_number || "—"}</td>
                  <td className="px-4 py-3 text-ink-500">{s.department}</td>
                  <td className="px-4 py-3 text-ink-500">{s.semester}</td>
                  <td className="px-4 py-3 text-ink-500">{s.section}</td>
                  <td className="px-4 py-3 text-right">
                    <button onClick={() => handleDelete(s.id, s.name)} className="text-ink-300 hover:text-rust p-1"><Trash2 size={15} /></button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}

      {showAdd && (
        <div className="fixed inset-0 bg-ink-900/40 flex items-center justify-center px-4 z-50">
          <div className="card w-full max-w-md p-6 bg-ivory">
            <div className="flex items-center justify-between mb-4">
              <h3 className="text-lg text-ink-800">Add Student</h3>
              <button onClick={() => setShowAdd(false)} className="text-ink-300 hover:text-ink-500"><X size={18} /></button>
            </div>
            <form onSubmit={handleAdd} className="space-y-3">
              <div><label className="label">Full name</label><input required className="input" value={form.name} onChange={(e) => setForm({ ...form, name: e.target.value })} /></div>
              <div><label className="label">Email</label><input required type="email" className="input" value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} /></div>
              <div><label className="label">Roll number</label><input className="input" value={form.rollNumber} onChange={(e) => setForm({ ...form, rollNumber: e.target.value })} /></div>
              <div><label className="label">Department</label><input required className="input" value={form.department} onChange={(e) => setForm({ ...form, department: e.target.value })} /></div>
              <div className="grid grid-cols-2 gap-3">
                <div><label className="label">Semester</label><input required type="number" min="1" max="8" className="input" value={form.semester} onChange={(e) => setForm({ ...form, semester: e.target.value })} /></div>
                <div><label className="label">Section</label><input className="input" value={form.section} onChange={(e) => setForm({ ...form, section: e.target.value })} /></div>
              </div>
              {error && <p className="text-sm text-rust">{error}</p>}
              <div className="flex gap-3 pt-2">
                <button type="submit" disabled={saving} className="btn-primary flex-1">{saving && <Loader2 size={16} className="animate-spin" />} Add Student</button>
                <button type="button" onClick={() => setShowAdd(false)} className="btn-secondary flex-1">Cancel</button>
              </div>
            </form>
          </div>
        </div>
      )}
    </Layout>
  );
}
```

CHAPTER 11: HOD & ADMINISTRATION PAGES

HOD PAGE

41. Faculty Appraisals & Scoring Engine Page

📁 Source File: `frontend/src/pages/hod/HodAppraisalsPage.jsx`

Appraisal generation panel to trigger calculations, review rankings, and inspect scores.

```jsx
import { useEffect, useState } from "react";
import { Award, Loader2, RefreshCcw } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";

const GRADE_STYLES = {
  Excellent: "bg-leaf/10 text-leaf",
  Satisfactory: "bg-gold-100 text-gold-700",
  "Needs Improvement": "bg-rust/10 text-rust",
};

export default function HodAppraisalsPage() {
  const [rows, setRows] = useState([]);
  const [loading, setLoading] = useState(true);
  const [generating, setGenerating] = useState(false);
  const [semester, setSemester] = useState(3);
  const [academicYear, setAcademicYear] = useState("2025-26");
  const [message, setMessage] = useState("");

  async function load() {
    setLoading(true);
    const facRes = await api.get("/faculty");
    const results = await Promise.all(
      facRes.data.data.map((f) =>
        api.get(`/appraisal/${f.id}`, { params: { semester, academicYear } }).then((r) => ({ ...r.data.data, facultyName: f.name, department: f.department }))
      )
    );
    setRows(results.sort((a, b) => b.appraisalScore - a.appraisalScore));
    setLoading(false);
  }

  useEffect(() => { load(); /* eslint-disable-next-line */ }, [semester, academicYear]);

  async function handleGenerate() {
    setGenerating(true);
    setMessage("");
    try {
      const res = await api.post("/appraisal/generate", { semester: Number(semester), academicYear });
      setMessage(res.data.message);
      load();
    } finally {
      setGenerating(false);
    }
  }

  return (
    <Layout title="Appraisals" subtitle="Generated automatically from submitted feedback using the configured formula">
      <div className="flex flex-wrap items-center gap-3 mb-6">
        <input className="input w-auto" value={academicYear} onChange={(e) => setAcademicYear(e.target.value)} placeholder="Academic Year" />
        <input className="input w-24" type="number" value={semester} onChange={(e) => setSemester(e.target.value)} placeholder="Sem" />
        <button onClick={handleGenerate} disabled={generating} className="btn-primary">
          {generating ? <Loader2 size={16} className="animate-spin" /> : <RefreshCcw size={16} />}
          Regenerate Appraisals
        </button>
        {message && <span className="text-sm text-leaf">{message}</span>}
      </div>

      {loading ? (
        <div className="flex items-center justify-center py-24 text-ink-400">
          <Loader2 className="animate-spin mr-2" size={18} /> Loading appraisals…
        </div>
      ) : rows.length === 0 ? (
        <div className="card flex flex-col items-center justify-center py-20 text-center">
          <Award size={28} className="text-ink-300 mb-3" />
          <p className="text-ink-600">No appraisal data for this period yet</p>
        </div>
```

```
      ) : (
        <div className="card overflow-x-auto">
          <table className="w-full text-sm">
            <thead>
              <tr className="border-b border-ink-100 text-left text-ink-400">
                <th className="px-4 py-3 font-medium">Faculty</th>
                <th className="px-4 py-3 font-medium">Department</th>
                <th className="px-4 py-3 font-medium">Responses</th>
                <th className="px-4 py-3 font-medium">Avg. Rating</th>
                <th className="px-4 py-3 font-medium">Appraisal Score</th>
                <th className="px-4 py-3 font-medium">Grade</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-ink-100">
              {rows.map((r) => (
                <tr key={r.facultyId} className="hover:bg-ink-50/50">
                  <td className="px-4 py-3 text-ink-700 font-medium">{r.facultyName}</td>
                  <td className="px-4 py-3 text-ink-500">{r.department}</td>
                  <td className="px-4 py-3 text-ink-500">{r.totalResponses}</td>
                  <td className="px-4 py-3 text-ink-500">{r.weightedAverageRating} / {r.scaleMax}</td>
                  <td className="px-4 py-3 text-ink-700 font-medium">{r.appraisalScore}%</td>
                  <td className="px-4 py-3">
                    <span className={`badge ${GRADE_STYLES[r.grade]}`}>{r.grade}</span>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </Layout>
  );
}

```

CHAPTER 11: HOD & ADMINISTRATION PAGES

HOD PAGE

42. Feedback Period & Criteria Settings

📁 Source File: `frontend/src/pages/hod/HodSettingsPage.jsx`

Controls active criteria, relative appraisal weights, and opening/closing semester feedback periods.

```jsx
import { useEffect, useState } from "react";
import { Loader2, Play, Square } from "lucide-react";
import api from "../../api/client";
import Layout from "../../components/Layout";

export default function HodSettingsPage() {
  const [criteria, setCriteria] = useState([]);
  const [config, setConfig] = useState(null);
  const [periods, setPeriods] = useState([]);
  const [loading, setLoading] = useState(true);
  const [newPeriod, setNewPeriod] = useState({ academicYear: "2025-26", semester: 3, allowEditing: false });
  const [savingConfig, setSavingConfig] = useState(false);

  function load() {
    setLoading(true);
    Promise.all([
      api.get("/admin/criteria"),
      api.get("/appraisal/config"),
      api.get("/admin/periods"),
    ]).then(([c, cfg, p]) => {
      setCriteria(c.data.data);
      setConfig(cfg.data.data);
      setPeriods(p.data.data);
      setLoading(false);
    });
  }
  useEffect(load, []);

  async function toggleCriterion(c) {
    await api.put(`/admin/criteria/${c.id}`, { isActive: c.is_active ? 0 : 1 });
    load();
  }

  async function saveConfig(e) {
    e.preventDefault();
    setSavingConfig(true);
    await api.put("/appraisal/config", {
      scaleMax: Number(config.scale_max),
      passThreshold: Number(config.pass_threshold),
      excellentThreshold: Number(config.excellent_threshold),
    });
    setSavingConfig(false);
  }

  async function openPeriod(e) {
    e.preventDefault();
    await api.post("/admin/periods/open", newPeriod);
    load();
  }

  async function closePeriod() {
    if (!window.confirm("Close feedback collection for the current period?")) return;
    await api.post("/admin/periods/close");
    load();
  }

  const activePeriod = periods.find((p) => p.is_open);

  if (loading) {
    return (
      <Layout title="Settings">
        <div className="flex items-center justify-center py-24 text-ink-400">
          <Loader2 className="animate-spin mr-2" size={18} /> Loading settings…
        </div>
      </Layout>
    );
  }

  return (
```

```
    <Layout title="Settings" subtitle="Configure evaluation criteria, appraisal formula and feedback periods">
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        <div className="card p-6">
          <h3 className="text-base text-ink-700 mb-1">Feedback Collection Period</h3>
          <p className="text-xs text-ink-400 mb-4">Start or stop the current semester's feedback window</p>

          {activePeriod ? (
            <div className="mb-4 bg-leaf/5 border border-leaf/20 rounded-lg px-4 py-3 flex items-center justify-between">
              <div>
                <p className="text-sm text-ink-700 font-medium">Currently open</p>
                <p className="text-xs text-ink-400">{activePeriod.academic_year} · Semester {activePeriod.semester}</p>
              </div>
              <button onClick={closePeriod} className="btn-danger"><Square size={14} /> Close</button>
            </div>
          ) : (
            <p className="mb-4 text-sm text-ink-400">No feedback period is currently open.</p>
          )}

          <form onSubmit={openPeriod} className="space-y-3 pt-3 border-t border-ink-100">
            <p className="text-sm font-medium text-ink-600">Open a new period</p>
            <div className="grid grid-cols-2 gap-3">
              <input className="input" placeholder="Academic Year" value={newPeriod.academicYear} onChange={(e) => setNewPe
riod({ ...newPeriod, academicYear: e.target.value })} />
              <input className="input" type="number" placeholder="Semester" value={newPeriod.semester} onChange={(e) => set
NewPeriod({ ...newPeriod, semester: Number(e.target.value) })} />
            </div>
            <label className="flex items-center gap-2 text-sm text-ink-600">
              <input type="checkbox" checked={newPeriod.allowEditing} onChange={(e) => setNewPeriod({ ...newPeriod, allowEd
iting: e.target.checked })} />
              Allow students to edit submitted feedback
            </label>
            <button type="submit" className="btn-secondary"><Play size={14} /> Open Period</button>
          </form>
        </div>

        <div className="card p-6">
          <h3 className="text-base text-ink-700 mb-1">Appraisal Formula</h3>
          <p className="text-xs text-ink-400 mb-4">appraisalScore = (weighted average rating ÷ scale max) × 100</p>
          <form onSubmit={saveConfig} className="space-y-3">
            <div>
              <label className="label">Rating scale maximum</label>
              <input type="number" step="0.1" className="input" value={config.scale_max} onChange={(e) => setConfig({ ...co
nfig, scale_max: e.target.value })} />
            </div>
            <div>
              <label className="label">Satisfactory threshold (avg. rating)</label>
              <input type="number" step="0.1" className="input" value={config.pass_threshold} onChange={(e) => setConfig({
...config, pass_threshold: e.target.value })} />
            </div>
            <div>
              <label className="label">Excellent threshold (avg. rating)</label>
              <input type="number" step="0.1" className="input" value={config.excellent_threshold} onChange={(e) => setConf
ig({ ...config, excellent_threshold: e.target.value })} />
            </div>
            <button type="submit" disabled={savingConfig} className="btn-primary">
              {savingConfig && <Loader2 size={16} className="animate-spin" />} Save Formula
            </button>
          </form>
        </div>

        <div className="card p-6 lg:col-span-2">
          <h3 className="text-base text-ink-700 mb-1">Evaluation Criteria</h3>
          <p className="text-xs text-ink-400 mb-4">Toggle which criteria appear on the student feedback form</p>
          <div className="divide-y divide-ink-100">
            {criteria.map((c) => (
              <div key={c.id} className="py-3 flex items-center justify-between">
                <span className="text-sm text-ink-600">{c.name}</span>
                <button
                  onClick={() => toggleCriterion(c)}
                  className={`badge cursor-pointer ${c.is_active ? "bg-leaf/10 text-leaf" : "bg-ink-100 text-ink-400"}`}
                >
                  {c.is_active ? "Active" : "Disabled"}
                </button>
              </div>
            ))}
          </div>
        </div>
      </div>
    </Layout>
  );
```

```
144 }
145
```

CHAPTER 11: HOD & ADMINISTRATION PAGES

HOD PAGE

43. Reports & CSV Export Page

📁 Source File: `frontend/src/pages/hod/HodReportsPage.jsx`

Export interface generating downloadable CSV appraisal reports for audits and college reviews.

```jsx
import { useState } from "react";
import { FileDown } from "lucide-react";
import Layout from "../../components/Layout";

export default function HodReportsPage() {
  const [semester, setSemester] = useState(3);
  const [academicYear, setAcademicYear] = useState("2025-26");

  function handleDownload() {
    const token = localStorage.getItem("ffas_token");
    const base = import.meta.env.VITE_API_URL || "http://localhost:5000/api";
    // Use a temporary link with the token as a query fallback isn't supported by our API (header-only auth),
    // so we fetch and trigger a blob download instead of a plain navigation.
    fetch(`${base}/admin/reports/appraisal.csv?semester=${semester}&academicYear=${encodeURIComponent(academicYear)}`, {
      headers: { Authorization: `Bearer ${token}` },
    })
      .then((res) => res.blob())
      .then((blob) => {
        const url = window.URL.createObjectURL(blob);
        const a = document.createElement("a");
        a.href = url;
        a.download = `appraisal_report_${academicYear}_sem${semester}.csv`;
        document.body.appendChild(a);
        a.click();
        a.remove();
        window.URL.revokeObjectURL(url);
      });
  }

  return (
    <Layout title="Reports" subtitle="Export faculty appraisal reports for a semester">
      <div className="card p-6 max-w-lg">
        <h3 className="text-base text-ink-700 mb-4">Faculty Appraisal Report (CSV)</h3>
        <p className="text-sm text-ink-500 mb-5">
          Includes faculty details, department, total responses, average rating, final appraisal score and grade for every faculty member.
        </p>
        <div className="grid grid-cols-2 gap-3 mb-5">
          <div>
            <label className="label">Academic Year</label>
            <input className="input" value={academicYear} onChange={(e) => setAcademicYear(e.target.value)} />
          </div>
          <div>
            <label className="label">Semester</label>
            <input type="number" className="input" value={semester} onChange={(e) => setSemester(e.target.value)} />
          </div>
        </div>
        <button onClick={handleDownload} className="btn-primary">
          <FileDown size={16} /> Download CSV Report
        </button>
      </div>
    </Layout>
  );
}

```

CHAPTER 11: HOD & ADMINISTRATION PAGES

AUTH PAGE

44. Unified Login & Student Self-Registration Page

📁 Source File: `frontend/src/pages/Login.jsx`

Tabbed portal allowing HOD, faculty, and students to log in, and new students to register.

```jsx
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { BookMarked, Loader2, UserPlus, LogIn } from "lucide-react";
import { useAuth } from "../context/AuthContext";

export default function Login() {
  const { login, register, error, loading, setError } = useAuth();
  const navigate = useNavigate();

  const [mode, setMode] = useState("login"); // "login" | "register"

  // Login form state
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  // Student registration form state
  const [regForm, setRegForm] = useState({
    name: "",
    email: "",
    rollNumber: "",
    department: "Information Technology",
    semester: 5,
    section: "A",
    password: "",
  });

  async function handleLoginSubmit(e) {
    e.preventDefault();
    try {
      const user = await login(email, password);
      navigate(`/${user.role}`);
    } catch {
      // error handled in context
    }
  }

  async function handleRegisterSubmit(e) {
    e.preventDefault();
    try {
      const user = await register(regForm);
      navigate(`/${user.role}`);
    } catch {
      // error handled in context
    }
  }

  return (
    <div className="min-h-screen bg-ink-800 flex items-center justify-center px-4 py-10">
      <div className="w-full max-w-md">
        <div className="flex flex-col items-center mb-6">
          <div className="h-12 w-12 rounded-xl bg-gold-500 flex items-center justify-center mb-3">
            <BookMarked size={24} className="text-ink-800" />
          </div>
          <h1 className="text-2xl text-ivory text-center font-serif">Faculty Feedback &amp;amp; Appraisal System</h1>
          <p className="text-ink-300 text-sm mt-1 text-center">
            {mode === "login"
              ? "Sign in to submit feedback, review performance, or manage analytics."
              : "Register as a student to provide semester feedback."}
          </p>
        </div>

        {/* Mode Toggle Buttons */}
        <div className="flex rounded-lg bg-ink-900/60 p-1 mb-5 border border-ink-700">
          <button
            type="button"
            onClick={() => {
              setError("");
              setMode("login");
            }}
```

```
            className={`flex-1 flex items-center justify-center gap-2 py-2 text-sm font-medium rounded-md transition-colors ${
              mode === "login"
                ? "bg-gold-500 text-ink-900 shadow-sm"
                : "text-ink-300 hover:text-white"
            }`}
          >
            <LogIn size={15} /> Sign In
          </button>
          <button
            type="button"
            onClick={() => {
              setError("");
              setMode("register");
            }}
            className={`flex-1 flex items-center justify-center gap-2 py-2 text-sm font-medium rounded-md transition-colors ${
              mode === "register"
                ? "bg-gold-500 text-ink-900 shadow-sm"
                : "text-ink-300 hover:text-white"
            }`}
          >
            <UserPlus size={15} /> Student Registration
          </button>
        </div>

        {mode === "login" ? (
          <form onSubmit={handleLoginSubmit} className="card p-6 space-y-4">
            <div>
              <label className="label" htmlFor="email">Email address</label>
              <input
                id="email"
                type="email"
                required
                className="input"
                placeholder="you@skit.ac.in"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                autoComplete="email"
              />
            </div>
            <div>
              <label className="label" htmlFor="password">Password</label>
              <input
                id="password"
                type="password"
                required
                className="input"
                placeholder="••••••••"
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                autoComplete="current-password"
              />
            </div>

            {error && (
              <p className="text-sm text-rust bg-rust/5 border border-rust/20 rounded-lg px-3 py-2">{error}</p>
            )}

            <button type="submit" disabled={loading} className="btn-primary w-full">
              {loading && <Loader2 size={16} className="animate-spin mr-2" />}
              Sign In
            </button>
          </form>
        ) : (
          <form onSubmit={handleRegisterSubmit} className="card p-6 space-y-3.5">
            <div>
              <label className="label" htmlFor="reg-name">Full Name</label>
              <input
                id="reg-name"
                type="text"
                required
                className="input"
                placeholder="Student Name"
                value={regForm.name}
                onChange={(e) => setRegForm({ ...regForm, name: e.target.value })}
              />
            </div>

            <div className="grid grid-cols-2 gap-3">
```

```
              <div>
                <label className="label" htmlFor="reg-email">College Email</label>
                <input
                  id="reg-email"
                  type="email"
                  required
                  className="input"
                  placeholder="b24xxxx@skit.ac.in"
                  value={regForm.email}
                  onChange={(e) => setRegForm({ ...regForm, email: e.target.value })}
                />
              </div>
              <div>
                <label className="label" htmlFor="reg-roll">Roll Number</label>
                <input
                  id="reg-roll"
                  type="text"
                  required
                  className="input"
                  placeholder="24ESKIT000"
                  value={regForm.rollNumber}
                  onChange={(e) => setRegForm({ ...regForm, rollNumber: e.target.value })}
                />
              </div>
            </div>

            <div className="grid grid-cols-3 gap-2">
              <div className="col-span-1">
                <label className="label" htmlFor="reg-dept">Department</label>
                <input
                  id="reg-dept"
                  type="text"
                  required
                  className="input text-xs"
                  value={regForm.department}
                  onChange={(e) => setRegForm({ ...regForm, department: e.target.value })}
                />
              </div>
              <div>
                <label className="label" htmlFor="reg-sem">Semester</label>
                <select
                  id="reg-sem"
                  className="input"
                  value={regForm.semester}
                  onChange={(e) => setRegForm({ ...regForm, semester: Number(e.target.value) })}
                >
                  {[1, 2, 3, 4, 5, 6, 7, 8].map((s) => (
                    <option key={s} value={s}>Sem {s}</option>
                  ))}
                </select>
              </div>
              <div>
                <label className="label" htmlFor="reg-sec">Section</label>
                <input
                  id="reg-sec"
                  type="text"
                  maxLength={2}
                  className="input"
                  value={regForm.section}
                  onChange={(e) => setRegForm({ ...regForm, section: e.target.value })}
                />
              </div>
            </div>

            <div>
              <label className="label" htmlFor="reg-password">Password</label>
              <input
                id="reg-password"
                type="password"
                required
                className="input"
                placeholder="Choose a password"
                value={regForm.password}
                onChange={(e) => setRegForm({ ...regForm, password: e.target.value })}
              />
            </div>

            {error && (
              <p className="text-sm text-rust bg-rust/5 border border-rust/20 rounded-lg px-3 py-2">{error}</p>
            )}
```

```
228
229             <button type="submit" disabled={loading} className="btn-primary w-full mt-2">
230               {loading && <Loader2 size={16} className="animate-spin mr-2" />}
231               Create Student Account &amp;amp; Sign In
232             </button>
233           </form>
234         )}
235
236         <div className="mt-4 text-center">
237           <p className="text-xs text-ink-400">
238             HOD Login: <span className="text-gold-400">hod@college.edu</span> | Password: <span className="text-gold-400">P
assword@123</span>
239           </p>
240         </div>
241       </div>
242     </div>
243   );
244 }
245
```

CHAPTER 12: BUILD & DEPENDENCY CONFIGURATION

CONFIG

45. Root Workspace Package Config

📁 Source File: `package.json`

Monorepo script definitions for concurrently running backend and frontend.

```
{
  "name": "faculty-feedback-system",
  "version": "1.0.0",
  "private": true,
  "description": "Faculty Feedback & Appraisal System",
  "scripts": {
    "install:all": "npm install --prefix backend && npm install --prefix frontend",
    "seed": "npm run seed --prefix backend",
    "clean": "npm run clean --prefix backend",
    "start:backend": "npm start --prefix backend",
    "dev:backend": "npm run dev --prefix backend",
    "start:frontend": "npm run dev --prefix frontend",
    "build:frontend": "npm run build --prefix frontend"
  },
  "engines": {
    "node": ">=18.0.0"
  }
}

```

CHAPTER 12: BUILD & DEPENDENCY CONFIGURATION

CONFIG

46. Backend Package & Dependencies

📁 Source File: `backend/package.json`

Backend runtime dependencies (better-sqlite3, express, bcryptjs, jsonwebtoken, cors).

```
{
  "name": "faculty-feedback-backend",
  "version": "1.0.0",
  "description": "Faculty Feedback & Appraisal System - Backend API",
  "main": "server.js",
  "type": "commonjs",
  "scripts": {
    "start": "node server.js",
    "start:backend": "node server.js",
    "dev": "node server.js",
    "seed": "node seed.js",
    "clean": "node clean.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "better-sqlite3": "^13.0.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "express-validator": "^7.2.0",
    "jsonwebtoken": "^9.0.2",
    "morgan": "^1.10.0"
  },
  "engines": {
    "node": ">=18.0.0"
  }
}

```

CHAPTER 12: BUILD & DEPENDENCY CONFIGURATION

CONFIG

47. Frontend Package & Dependencies

📁 Source File: `frontend/package.json`

Frontend build dependencies (react, vite, tailwindcss, recharts, lucide-react).

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "start": "vite",
    "start:frontend": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.20.0",
    "lucide-react": "^1.48.0",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.4",
    "recharts": "^3.10.1"
  },
  "devDependencies": {
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "autoprefixer": "^10.6.1",
    "oxlint": "^1.81.0",
    "postcss": "^8.5.28",
    "tailwindcss": "^3.4.19",
    "vite": "^8.3.0"
  },
  "engines": {
    "node": ">=18.0.0"
  }
}

```

CHAPTER 12: BUILD & DEPENDENCY CONFIGURATION

CONFIG

48. Vite Bundler Configuration

📁 Source File: `frontend/vite.config.js`

Vite build settings and React plugin configuration.

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})

```

CHAPTER 12: BUILD & DEPENDENCY CONFIGURATION

CONFIG

49. Tailwind CSS Design System Config

📁 Source File: `frontend/tailwind.config.js`

Tailwind CSS theme extensions, typography settings, and custom color definitions.

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,jsx}"],
  theme: {
    extend: {
      colors: {
        ink: {
          50: "#EEF1F7",
          100: "#D9E0EC",
          200: "#B3C1D9",
          300: "#7E92B8",
          400: "#4C6291",
          500: "#2F4373",
          600: "#25355C",
          700: "#1B2A4A",
          800: "#141F38",
          900: "#0E1628",
        },
        gold: {
          50: "#FBF4E4",
          100: "#F4E3BC",
          300: "#DDB165",
          500: "#C08A28",
          600: "#A5731F",
          700: "#835A17",
        },
        paper: "#F7F6F2",
        ivory: "#FCFBF8",
        leaf: "#3F7D5C",
        rust: "#B4483B",
      },
      fontFamily: {
        serif: ['"Source Serif 4"', "Georgia", "serif"],
        sans: ['"Inter"', "system-ui", "sans-serif"],
      },
      boxShadow: {
        card: "0 1px 2px rgba(14,22,40,0.06), 0 1px 0 rgba(14,22,40,0.04)",
      },
    },
  },
  plugins: [],
}


```